# Team Collaboration Platform

Vilkai – Technical Specification

Pijus Čepaitis, Rokas Braidokas, Augustinas Punis, Artūras Semenčiuk

## Contents

**1 Introduction** **3**

1.1 Purpose and problem statement . . . 3

1.2 Scope . . . 3

**2 Business Flows** **4**

2.1 Registration and login . . . 4

2.2 Team creation and joining via invite . . . 6

2.2.1 Description . . . 6

2.2.2 User flow diagram - Team creation . . . 7

2.2.3 User flow diagram - Joining with invite code . . . 8

2.2.4 Error cases . . . 8

2.3 Member and role management . . . 9

2.4 Task creation, assignment, and status changes . . . 11

2.5 Channel creation and membership (public/private) . . . 14

2.5.1 User flow diagram - Creating a channel . . . 14

2.6 Sending, editing, deleting, and moderating messages . . . 16

2.6.1 User flow diagram - Sending messages . . . 16

2.6.2 User flow diagram - Message editing . . . 18

2.6.3 User flow diagram - Deleting a message . . . 18

2.7 Presence and typing indicators . . . 19

2.7.1 User typing - Sequence diagram . . . 19

2.7.2 User presence - Sequence diagram . . . 21

2.8 System messages (join, leave, promote, task updates) . . . 22

2.8.1 When system messages appear . . . 22

2.8.2 Rules . . . 23

2.9 Replies . . . 23

2.10 Reactions and pinned messages . . . 25

2.11 Mentions and notifications . . . 26

2.12 Unread indicators and message search . . . . . 29
2.13 Push notifications . . . . . 30

**3 Architecture** **33**
3.1 Package diagram (high-level architecture) . . . . . 33
3.2 Technology stack . . . . . 34

**4 LoFi wireframes** **35**

**5 Data model** **36**
5.1 Complete data model diagram . . . . . 36
5.2 User, team, TeamMember, Invite, Task . . . . . 37
5.3 Channel, ChannelMember, Message . . . . . 40
5.4 Reaction, Mention, Notification, ReadState . . . . . 42

**6 Permissions matrix** **43**
6.1 Role x action matrix (owner/admin/member) . . . . . 43

**7 API contract** **45**
7.1 WebSocket - Connection, authentication, and event format . . . . . 45
7.2 WebSocket - Messaging, presence and typing events . . . . . 48
7.3 WebSocket - Reaction, notification and task events . . . . . 50

**8 Open questions and review** **51**

# 1 Introduction

## 1.1 Purpose and problem statement

University students work in project groups for most of their courses, but the tools they use are fragmented. Communication happens in Messenger or Discord, task assignment is often informal or missing entirely, and deadlines are kept in personal notes. As a result, responsibilities are unclear and information is scattered across several apps.

This system provides a workspace where the team is the primary entity. Real-time communication, task assignment and a simple personal deadline calendar all operate on the same shared team and membership model. Because roles, channels and members belong to one system, permissions follow team roles, private channels stay in sync with membership changes, and the system posts messages automatically when someone joins, leaves, is promoted or has a task updated.

## 1.2 Scope

The scope is defined using MoSCoW prioritization.

**In scope: Must have**

- Registration and login (JWT)
- Email verification
- Creating teams and joining them with an invite code (valid for one week)
- Team roles (Owner, Admin, Member) and their permissions
- Public and private channels
- Real-time messaging, including editing (at most 5 edits) and deleting messages (shown as "Message deleted")
- Team task board, task creation, assignment and the full status workflow
- Main system messages (join, leave, promote, task updates)

**In scope: Should have**

- Replies to messages, with a visible link to the original
- Mentions (`@tag`, `@channel`) and notifications
- Emoji reactions
- Pinned messages
- Unread indicators, typing indicators and online/offline status
- Personal notifications about task status changes
- Message search

**In scope: Could have**

- Personal calendar: each user sees only their own tasks and deadlines. It does not show other team members' availability.
- Native mobile application for Android and iOS with the same features as the web client, except team settings (web only). Push notifications for mentions, replies and task updates.
- Availability statuses (Available, Busy, Do not disturb)

**Won't have**

- Voice and video calls
- File attachments
- Threads (a reply links to the original message, but there is no separate thread view)
- Team calendar or shared schedule, including seeing other members' free time

- Message editing history

# 2 Business Flows

## 2.1 Registration and login

A visitor registers with an email, a tag, an optional display name and a password. The tag is a unique handle used for @mentions and for finding people (like Discord usernames); the display name is what others see next to messages and does not have to be unique. The account cannot be used until the email address is confirmed: the system sends a verification link, and the user can log in only after opening it. Logging in with email and password returns a short-lived access token and a refresh token (see Authentication). The web and mobile clients use the same endpoints. After login the user sees their team list; a user with no teams sees the Create team and Join team buttons.

1. The visitor fills in email, tag, display name (optional) and password.
2. The server checks the fields, creates the account and sends a verification email. No tokens are returned yet.
3. The user opens the link in the email; the account is confirmed and the user is logged in.
4. Later the user logs in with email and password and receives an access token and a refresh token.

**Account fields**

| Field | Format | Unique | Used for | Visible to other users |
|---|---|---|---|---|
| Email | Valid address per RFC 5322 (addr-spec; no comments or quoted local parts), max 254 characters (RFC 5321 limit); spaces at the start and end are trimmed; stored as entered | Yes, case-insensitive | Login, verification email, account emails | No |
| Tag | 3–32 characters: a–z, 0–9, underscore and period; cannot start or end with a period, no two periods in a row; uppercase input is converted to lowercase. Shown with @ (e.g. `@pijus_c`). Reserved tags: see Tag and display name rules | Yes, across the whole system | @mentions, finding people | Yes, as @tag: member list, profile card, mention suggestions |
| Display name | 1–32 characters after trimming; any letters (also ą, č, ę…), digits, spaces, punctuation and emoji; no line breaks or control characters. If left empty at registration, the tag is used | No | Showing who wrote a message or did an action | Yes: messages, member list, notifications |
| Password | 8–128 characters | - | Login | Never; stored only as a hash (ASP.NET Core Identity) |

Email is compared case-insensitively because, although RFC 5321 allows a case-sensitive local part,

almost all providers ignore case, and this prevents duplicate accounts.

**Tag and display name rules**

| Rule | Value |
|---|---|
| Reserved tags | `channel` (used by @channel), `here`, `everyone`, `admin`, `system` |
| Name filter | Tag and display name are checked against a blocklist of offensive words, case-insensitively and also inside the name, after replacing common look-alikes (0→o, 1→i, 3→e, 4→a, 5→s). A match returns `400 VALIDATION_ERROR` with issue "Name is not allowed" |
| Changing the display name | Possible at any time in Profile settings |
| Changing the tag | Not possible: the tag is permanent. |

**Email verification**

| Rule | Value |
|---|---|
| After registration | The account is created with `emailVerifiedAt = null` and a verification email is sent. The user is not logged in and receives no tokens |
| Verification link | Valid for 24 hours and can be used once. Opening it confirms the email and logs the user in |
| Resending | At most 3 times per hour; a new link makes older links invalid |
| Unverified account | Accounts not verified within 7 days are deleted, so the email and tag become free again |

**Login and logout**

| Rule | Value |
|---|---|
| Login | Email and password; returns an access token (15 min) and a refresh token (30 days), see Authentication |
| Not verified yet | Login with the correct password returns `403 EMAIL_NOT_VERIFIED`, and the client offers "Resend email" |
| Failed login | Wrong email or wrong password returns the same error, so it does not reveal which one was wrong |
| Too many attempts | After 10 failed attempts within 10 minutes: `429 TOO_MANY_REQUESTS` |
| Logout | Revokes the refresh token and deletes the device's push token (mobile) |

**Errors**

| Situation | Result |
|---|---|
| Field missing or invalid | `400 VALIDATION_ERROR` with details |
| Email or tag already used | `409 EMAIL_TAKEN` / `409 TAG_TAKEN` |
| Wrong email or password | `401 INVALID_CREDENTIALS` |
| Login before the email is verified (correct password) | `403 EMAIL_NOT_VERIFIED` |

| Situation | Result |
|---|---|
| Verification link invalid or already used | `400 INVALID_VERIFICATION_TOKEN` |
| Verification link expired | `410 VERIFICATION_TOKEN_EXPIRED`; the client offers "Resend email" |
| Too many attempts | `429 TOO_MANY_REQUESTS` |

## 2.2 Team creation and joining via invite

### 2.2.1 Description

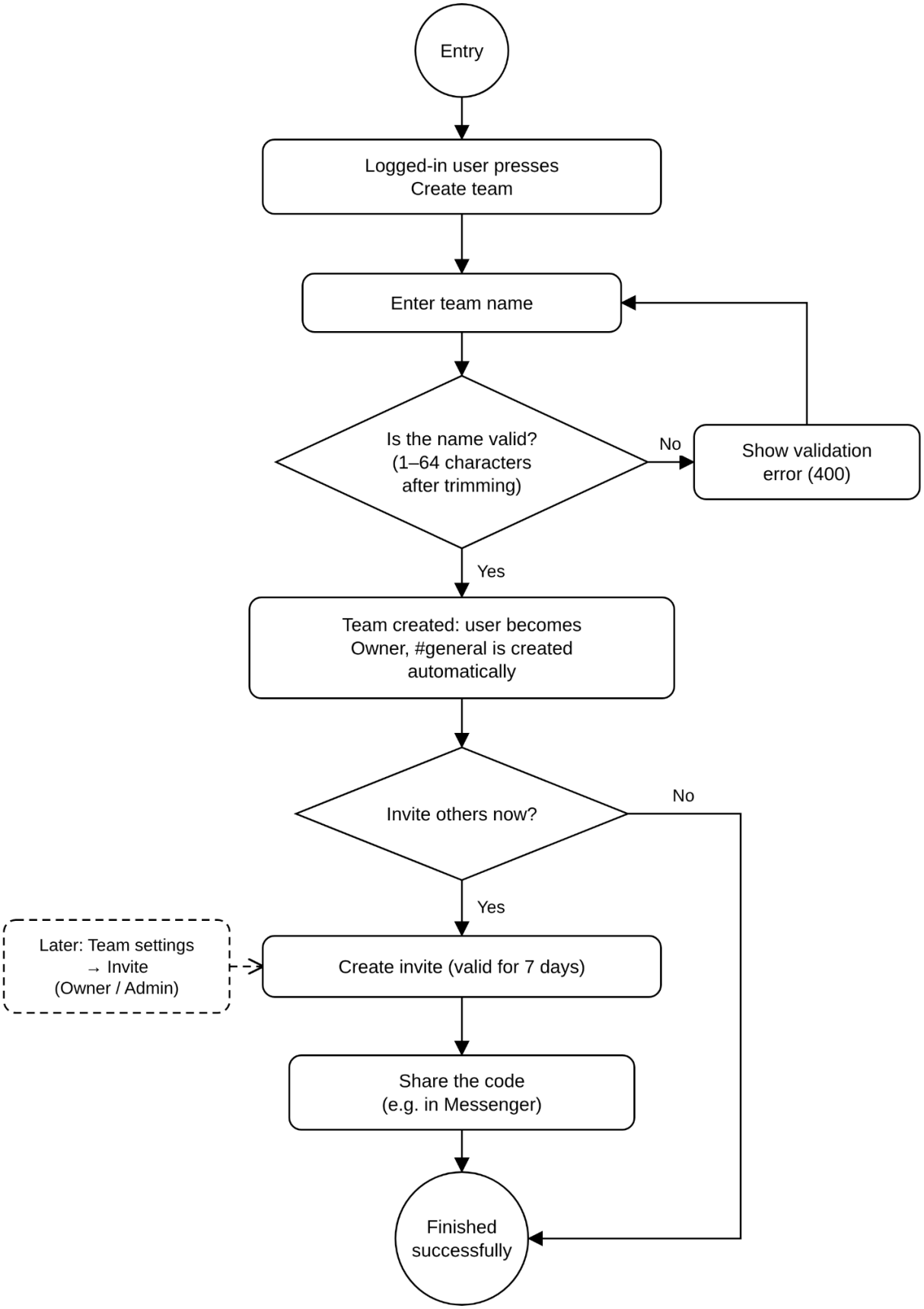
A team is the top-level workspace of the system. Any registered user can create a team and automatically becomes its Owner. Other users join the team only through an invite created by the Owner or an Admin.

Users join a team by entering the invite code in the Join team screen (web and mobile). The code is case-insensitive; spaces and hyphens are ignored, so `k7q2-m9xa` is accepted as `K7Q2M9XA`. After 10 failed join attempts within 10 minutes the user receives `429 TOO_MANY_REQUESTS`.

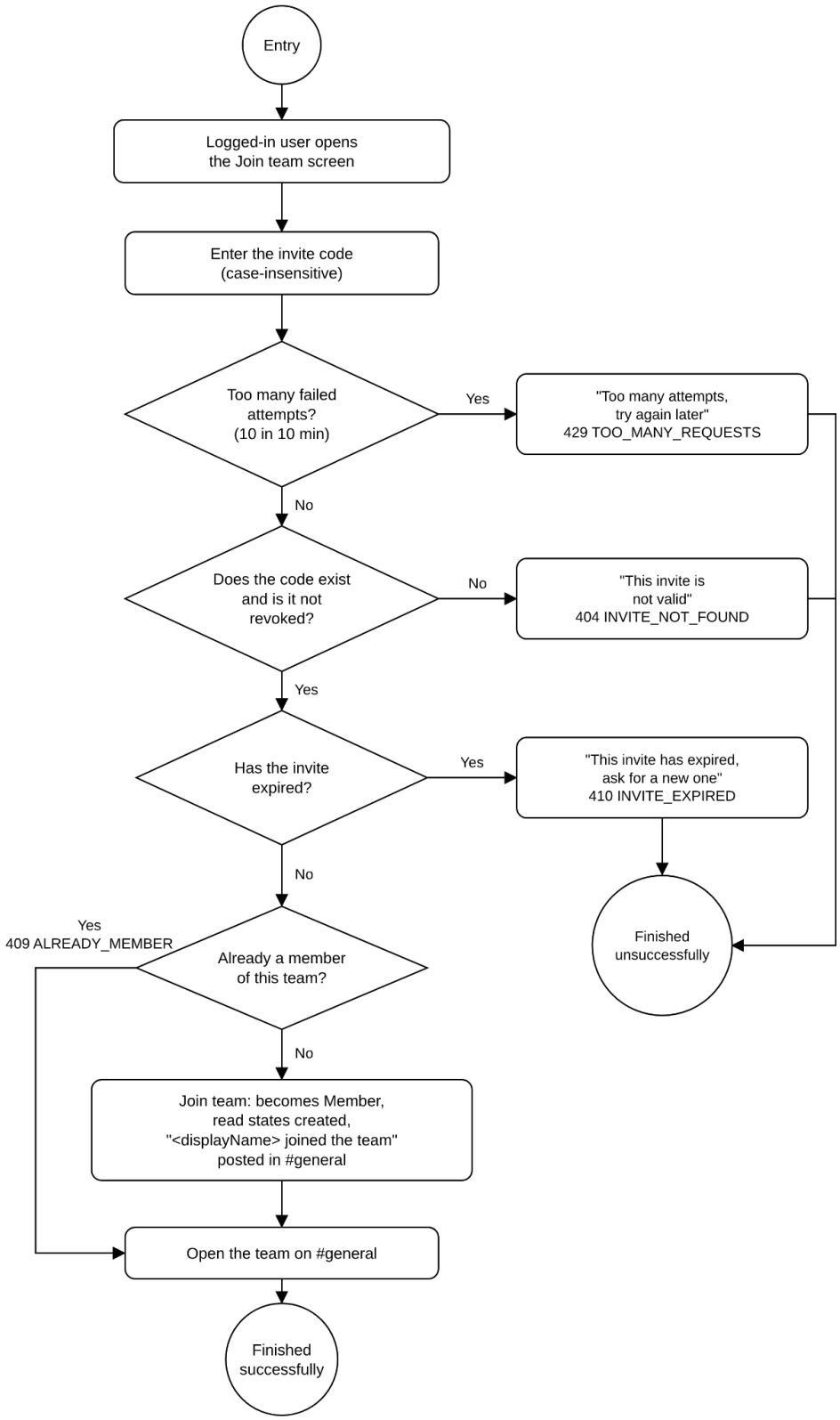
**Rules**

| Rule | Value |
|---|---|
| Who can create a team | Any authenticated user |
| Team name | 1-64 characters after trimming; not unique |
| Role of the creator | Owner. A team always has exactly one Owner, so one person is always responsible for the team (deleting it, managing roles) and permission checks stay simple. Ownership can be transferred, and an inactive Owner can be replaced (see Member and role management) |
| Created automatically with the team | `#general` public channel |
| Who can create an invite | Owner, Admin |
| Invite format | Short code (e.g. `K7Q2M9XA`) |
| Invite lifetime | 7 days from creation |
| Invite usage | Multi-use: any number of users can join with the same invite until it expires or is revoked |
| Revoking an invite | Owner, Admin; a revoked invite stops working immediately |
| Role of a user who joins | Member |
| Channels after joining | Member of `#general`; can view other public channels, private channels after being added |
| System message | "@<tag> joined the team" posted in `#general`; all online members see it immediately (`message.created`) and the member list updates (`member.joined`) |
| Code input | Case-insensitive; spaces and hyphens ignored |

### 2.2.2 User flow diagram - Team creation



### 2.2.3 User flow diagram - Joining with invite code




### 2.2.4 Error cases

| Situation | Result |
|---|---|
| Team name empty or longer than 64 characters | `400 VALIDATION_ERROR`, field `name` |
| Member (not Owner/Admin) tries to create an invite | `403 FORBIDDEN`; the button is hidden in the UI |
| Code does not exist or was revoked | `404 INVITE_NOT_FOUND` - "This invite is not valid" |
| Invite has expired | `410 INVITE_EXPIRED` - "This invite has expired, ask for a new one" |
| User is already a member of the team | `409 ALREADY_MEMBER`; the frontend simply opens the team |
| User is not logged in | `401 UNAUTHORIZED`; the Join team screen is available only after login |
| 10 failed join attempts within 10 minutes | `429 TOO_MANY_REQUESTS` - "Too many attempts, try again later" |

### 2.3 Member and role management

The Owner manages the team in Team settings (web only): promotes Members to Admin, demotes Admins, removes members and can transfer ownership to an Admin. Admins and Members can leave the team themselves. Every change is applied on the server in one transaction and then pushed to all online members, so nobody has to refresh: the member list, the visible controls and the channel list update immediately.

| Rule | Value |
|---|---|
| Promote / demote | Owner only; `MEMBER <-> ADMIN` |
| Transfer ownership | Owner only, to an Admin of the team, after typing the team name to confirm. In one transaction the Admin becomes `OWNER` and the old Owner becomes `ADMIN`. System message "@<tag> is now the Owner of the team" (tag of the new Owner); `member.role_changed` is sent for both users |
| Inactive Owner | If the Owner has not opened the app for 30 days (`User.lastSeenAt`), the Admin who has been in the team longest can claim ownership. It is applied at once like a transfer (same `member.role_changed` events and the same system message "@<tag> is now the Owner of the team", which does not say that anyone transferred anything), and the old Owner gets an email. If the team has no Admins, the longest-serving Member can claim it |
| Remove member | Owner only; cannot remove themselves |
| Leave team | Admin, Member. The Owner must first transfer ownership (and then can leave as Admin) or delete the team |
| Delete team | Owner only, after typing the team name to confirm; deletes all channels, messages, tasks and invites (CASCADE) |
| Leaving or removal | In the same transaction: `TeamMember`, `ChannelMember`, `ReadState` and `Notification` rows of that user in this team are deleted; their unfinished tasks (not DONE) become unassigned with status OPEN. Their messages stay, with their display name |

| Rule | Value |
|---|---|
| Access cut | Before the system message is posted, all the user's connections are removed from the `team-{teamId}` and `channel-{channelId}` groups, so they do not receive it or any later event |
| Rejoining | Possible with any valid invite of the team (also the same one, until it expires or is revoked); the user joins as Member with fresh read states |
| System messages (in `#general`) | "@<tag> joined the team", "@<tag> left the team", "@<tag> was removed from the team", "@<tag> was promoted to Admin", "@<tag> was demoted to Member", "@<tag> is now the Owner of the team" (also posted when ownership is claimed). The text uses the tag, as defined in System messages. Each system message is saved in the same transaction as the change and published after the commit |
| Role change takes effect | Immediately: the server checks the role on every request, and the client shows or hides Owner/Admin controls on `member.role_changed` |

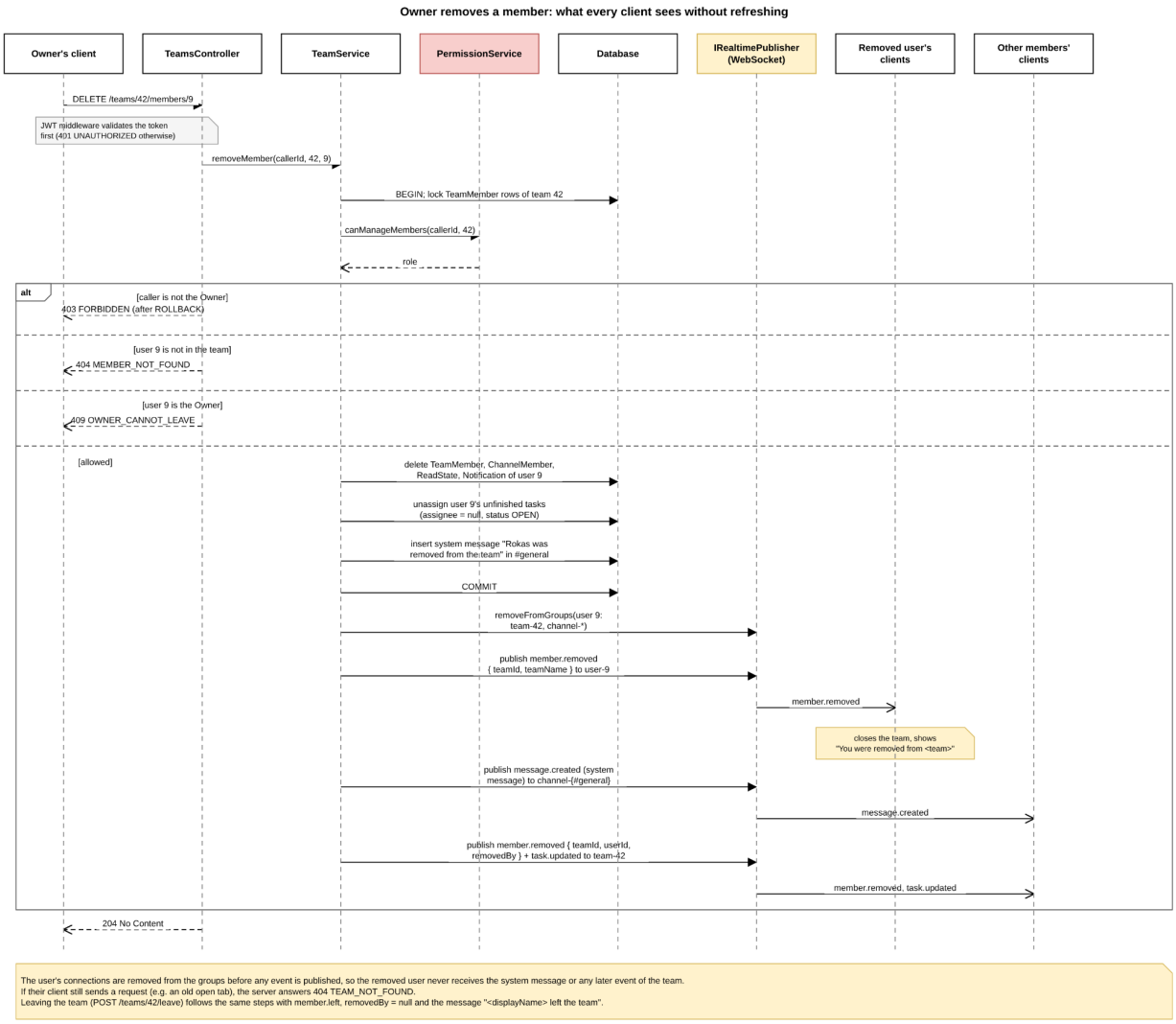
WebSocket events (sent by the server after the transaction commits; the client never sends them):

| Event (group) | Payload | What the client does |
|---|---|---|
| `member.joined` (team) | `teamId`, `member { id, tag, displayName, role, status, joinedAt }` | Adds the user to the member list |
| `member.left` / `member.removed` (team) | `teamId`, `userId`, `removedBy` (null on leave) | Removes the user from the list; unassigned tasks arrive as `task.updated` |
| `member.removed` (`user-{userId}`) | `teamId`, `teamName` | The removed user's client closes the team and shows "You were removed from <teamName>" |
| `member.role_changed` (team) | `teamId`, `userId`, `role`, `previousRole`, `changedBy` | Updates the role badge; for the affected user shows or hides admin controls |
| `team.updated` (team) | `teamId`, `name` | Updates the team name everywhere |
| `team.deleted` (team) | `teamId` | All clients leave the team and open the team list with "The team was deleted" |

| Situation | Result |
|---|---|
| Admin or Member tries to change roles or remove | `403 FORBIDDEN`; controls are hidden in the UI |
| Owner tries to leave, or to remove or demote themselves | `409 OWNER_CANNOT_LEAVE` - "Transfer ownership first" |
| Ownership transferred to someone who is not an Admin | `409 TARGET_NOT_ADMIN` |
| Claim while the Owner was active in the last 30 days | `409 OWNER_ACTIVE` |
| Claim by someone other than the longest-serving Admin (or Member) | `403 FORBIDDEN` |
| Target user is not in the team | `404 MEMBER_NOT_FOUND` |
| Role is already the requested one | `200 OK`, no change, no event |

| Situation | Result |
|---|---|
| Removed user still has the channel open and sends a message | `404 TEAM_NOT_FOUND`; the client shows the removal message |

### Sequence diagram - Removing a member

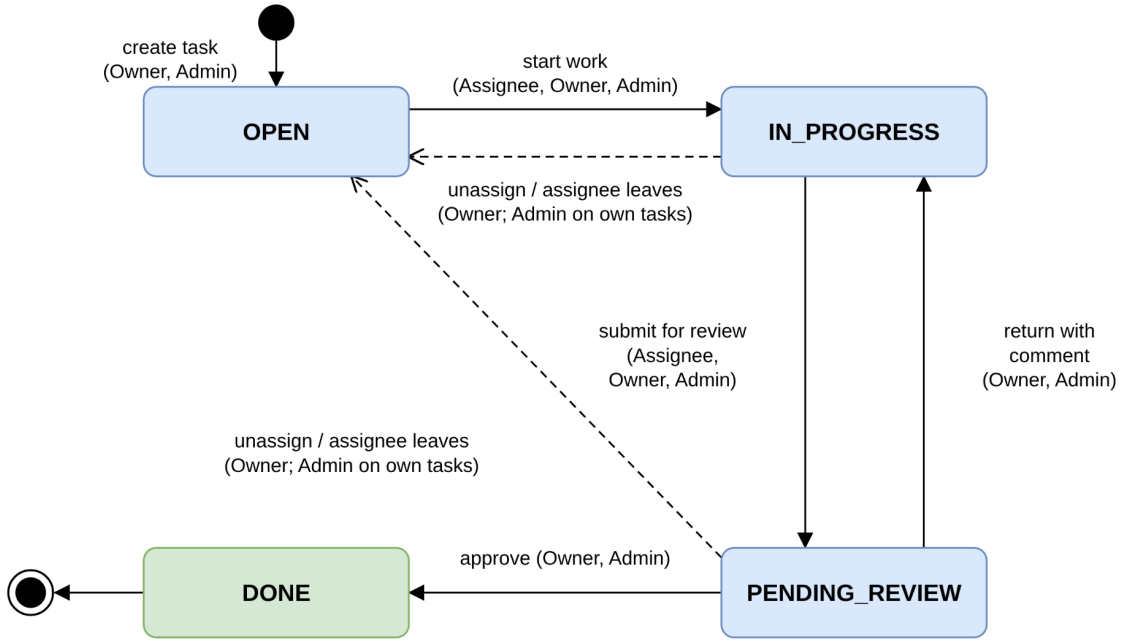



## 2.4 Task creation, assignment, and status changes

Owners and Admins create tasks on the team's task board, optionally assign them, and approve finished work. The assignee moves the task through the workflow. The board has one column per status and updates live for every member through `task.created` and `task.updated` (see WebSocket - Reaction, notification and task events), so a status change is visible without refreshing. Each change also posts a system message (see System messages) and notifies the creator and assignee (`TASK_ASSIGNED`, `TASK_STATUS_CHANGED`).

### State diagram - Task status workflow

**Task status workflow**



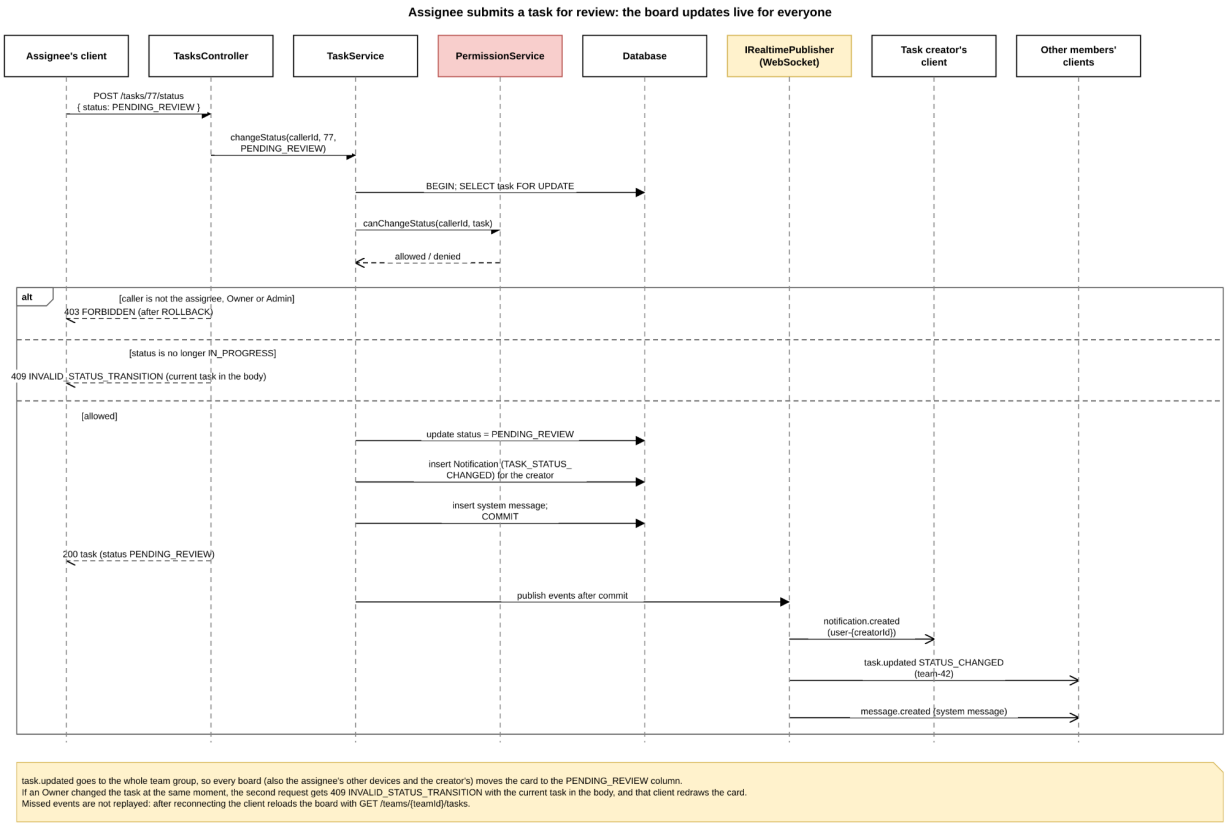

Assign and reassign (Owner, Admin) and take (any member, task OPEN and unassigned) do not change the status. "Admin on own tasks" = tasks the Admin created.
DONE is final. Every change: task.updated to team-{teamId}, a system message and a notification.
Change from an old status: 409 INVALID_STATUS_TRANSITION, the client refreshes the card.

| From | To | Who | Notes |
|---|---|---|---|
| (new) | `OPEN` | Owner, Admin | Assignee optional |
| `OPEN` | `IN_PROGRESS` | Assignee, Owner, Admin | Task must have an assignee |
| `IN_PROGRESS` | `PENDING_REVIEW` | Assignee, Owner, Admin | Assignee submits for review |
| `PENDING_REVIEW` | `DONE` | Owner, Admin | Final status; a DONE task cannot change any more |
| `PENDING_REVIEW` | `IN_PROGRESS` | Owner, Admin | Comment required (1-1000 characters), shown on the task as `reviewComment` |
| any except `DONE` | `OPEN`, no assignee | Owner; Admin only on tasks they created | Unassigning; also done automatically when the assignee leaves the team |

| Rule | Value |
|---|---|
| Fields | `title` 1-120 characters, `description` 0-2000, `dueDate` optional (`YYYY-MM-DD`) |
| Assign | Owner: to anyone in the team. Admin: to Members, only on tasks they created. Reassigning keeps the status |
| Take an open task | Any member, if the task is OPEN and has no assignee |
| Abandon | The assignee cannot unassign themselves; they ask an Owner or Admin |
| Edit / delete | Owner, or the Admin who created the task; not possible when DONE (edit) |
| Simultaneous changes | The task row is locked (`SELECT ... FOR UPDATE`). If two users take the same task, the second gets `409 TASK_ALREADY_ASSIGNED`. A transition from a status that has already changed returns `409 INVALID_STATUS_TRANSITION` with the current task, and the client refreshes the card |

| Situation | Result |
|---|---|
| Title empty or too long, bad date | `400 VALIDATION_ERROR` |
| Role or assignee does not allow the action | `403 FORBIDDEN` |
| Task missing or in another team | `404 TASK_NOT_FOUND` |
| Assignee is not a team member | `404 MEMBER_NOT_FOUND` |
| Transition not allowed from current status | `409 INVALID_STATUS_TRANSITION` |
| Taking a task that already has an assignee | `409 TASK_ALREADY_ASSIGNED` |
| Editing or (un)assigning a DONE task | `409 TASK_ALREADY_DONE` |

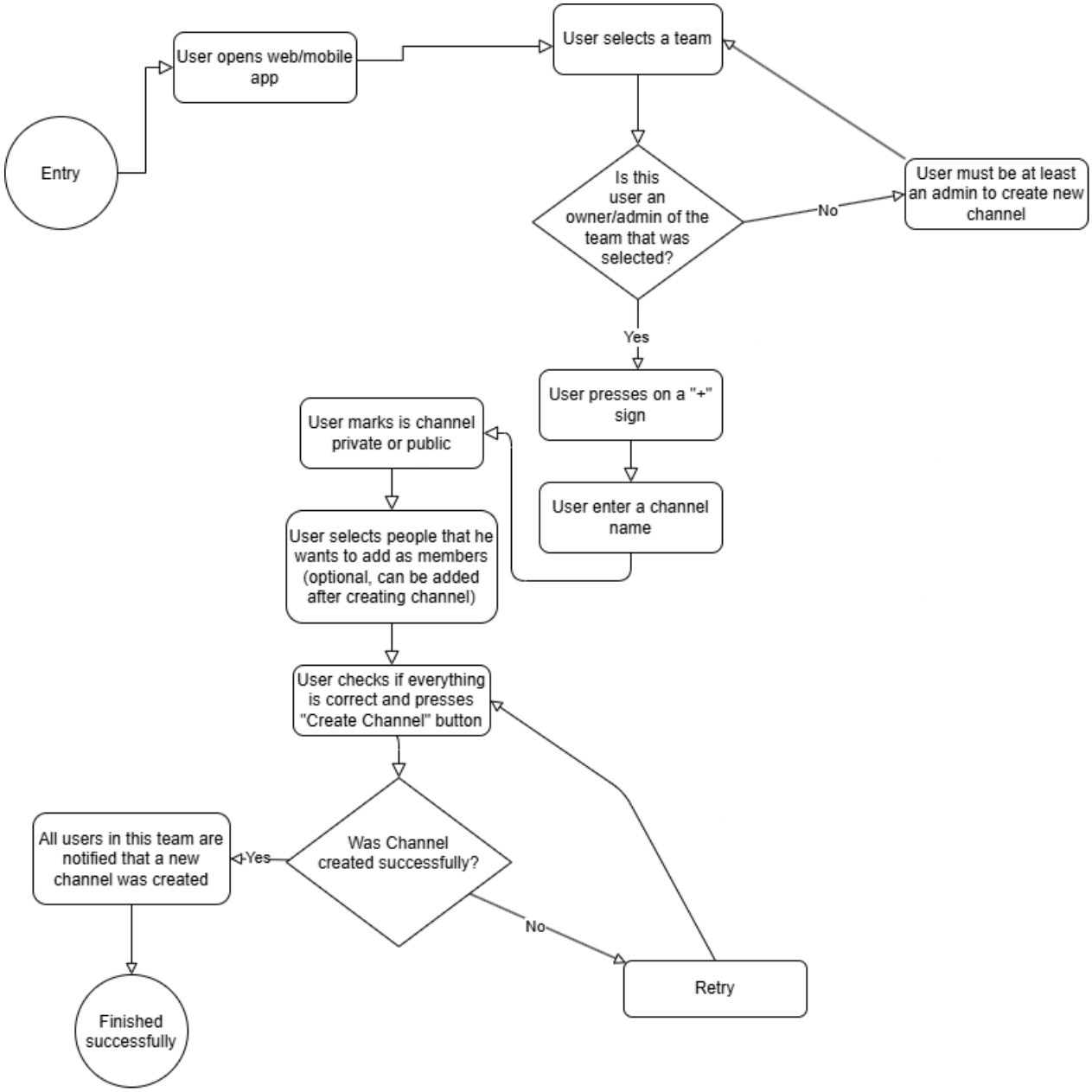
**Sequence diagram - Task status change in real time**




## 2.5 Channel creation and membership (public/private)

### 2.5.1 User flow diagram - Creating a channel

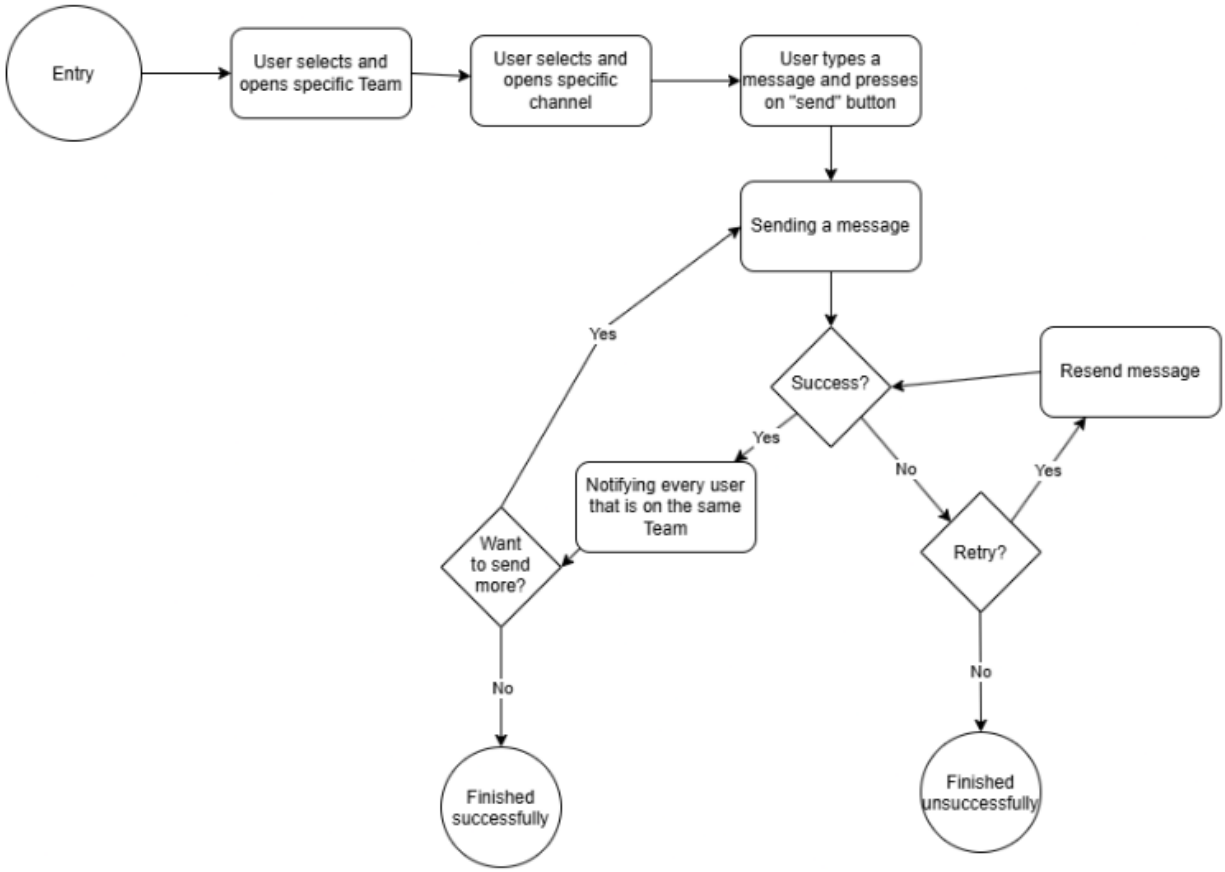
The flow covers creating a new channel in an existing Team. Worth mentioning that the user who wants to create a channel must have an admin or owner role to successfully create a channel. Notification only goes to all the users that are in the channel (e.g. if the channel is private).


Entry
User opens web/mobile app
User selects a team
Is this user an owner/admin of the team that was selected?
No
User must be at least an admin to create new channel
Yes
User presses on a "+" sign
User enter a channel name
User marks is channel private or public
User selects people that he wants to add as members (optional, can be added after creating channel)
User checks if everything is correct and presses "Create Channel" button
Was Channel created successfully?
Yes
All users in this team are notified that a new channel was created
No
Retry
Finished successfully

## 2.6 Sending, editing, deleting, and moderating messages

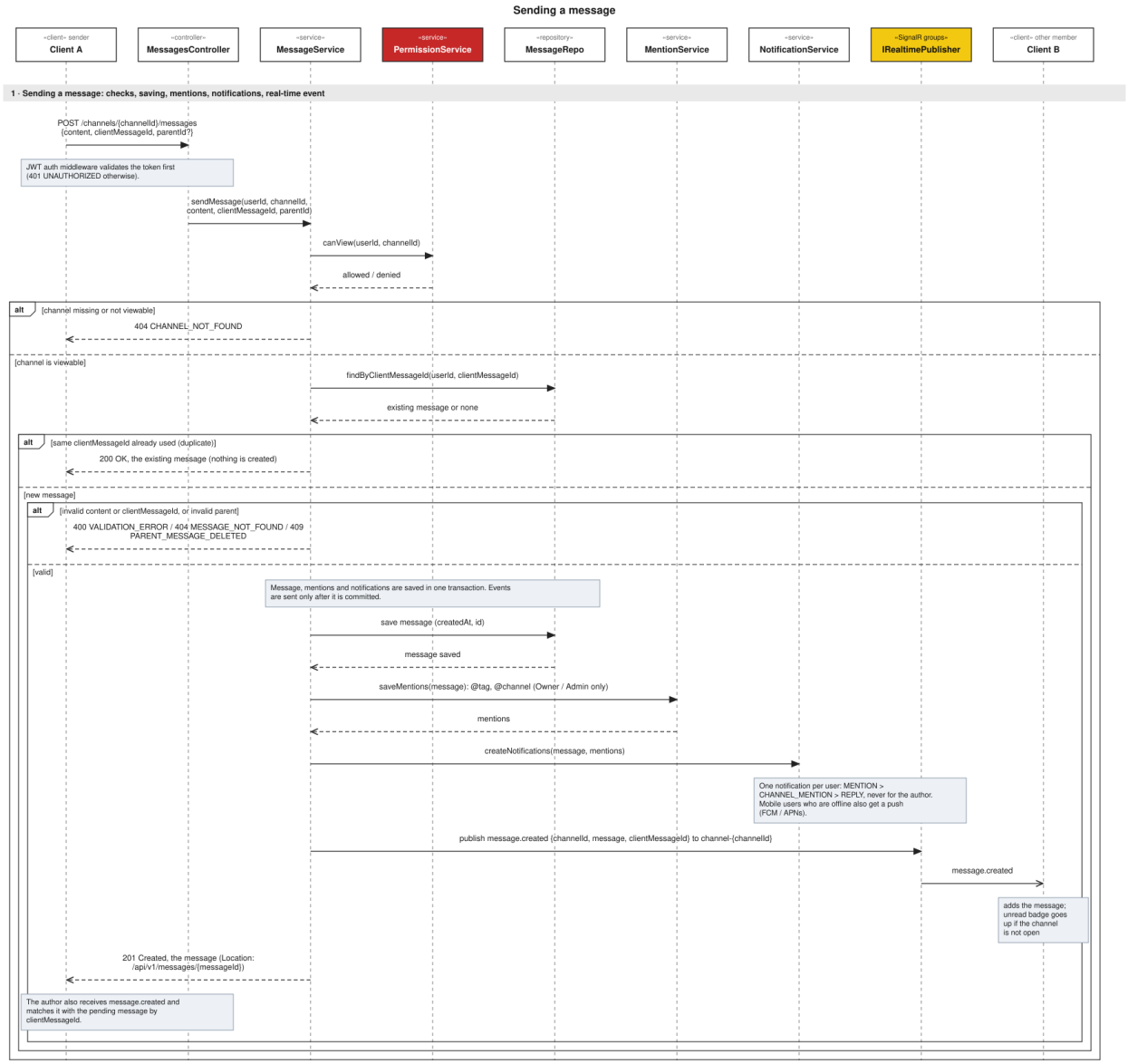
### 2.6.1 User flow diagram - Sending messages

The flow covers sending a message in a team chat app. The user selects a specific Team and channel. We assume that this user can send a message to the channel he selected earlier. Then the User sends the message, if it was sent successfully, the system notifies other users that can see this channel that a new message has appeared in “Team name” “Channel name”. Retry happens because of the network error, and the user retries with the same `clientMessageId`.



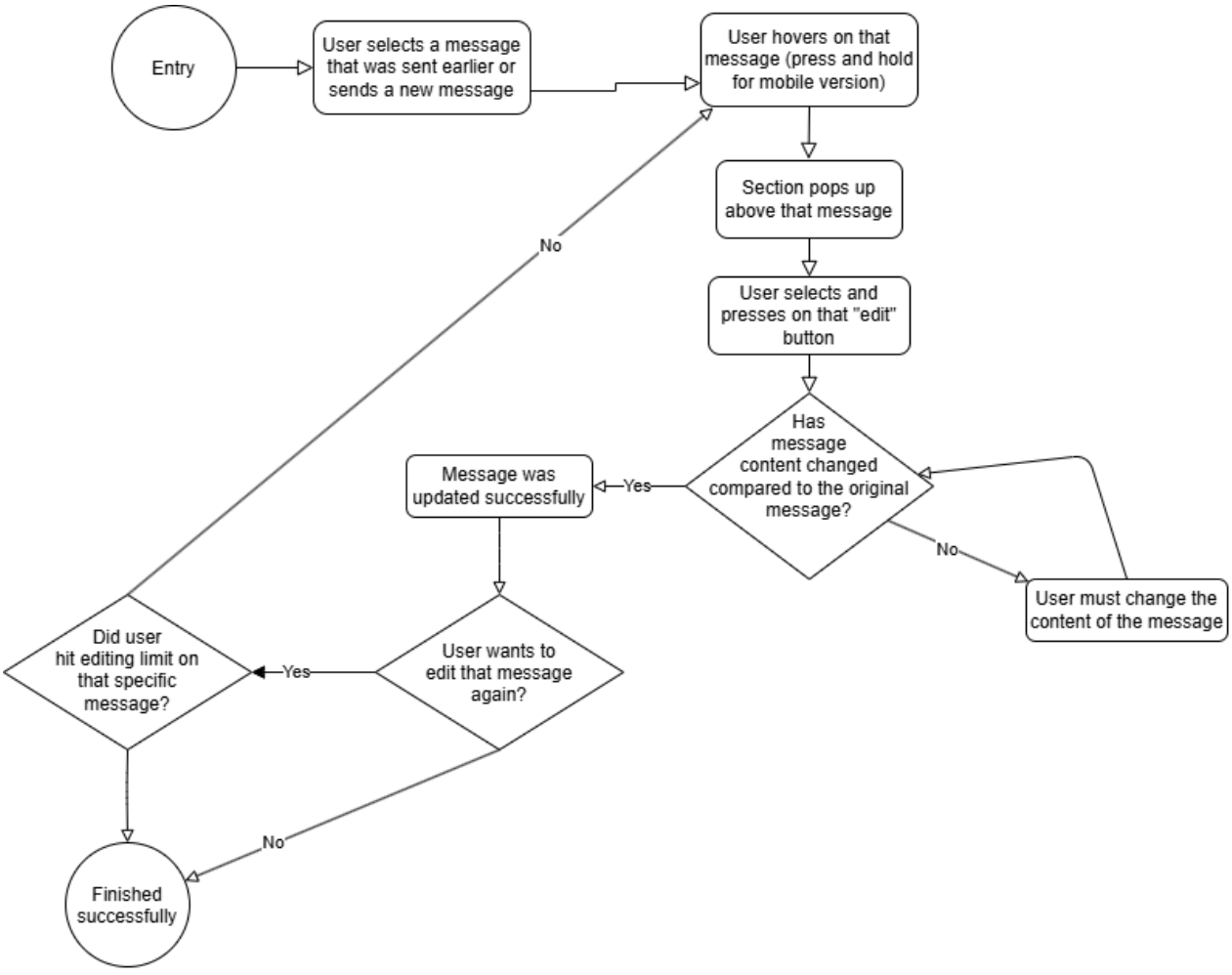

### Sequence diagram - Sending messages

If the response is lost, the client resends the same request with the same `clientMessageId`. The server answers `200 OK` with the existing message (the duplicate branch above), so no second message appears.


Sending a message
«client» sender
Client A
«controller»
MessagesController
«service»
MessageService
«service»
PermissionService
«repository»
MessageRepo
«service»
MentionService
«service»
NotificationService
«SignalR groups»
IRealtimePublisher
«client» other member
Client B
1 · Sending a message: checks, saving, mentions, notifications, real-time event
POST /channels/{channelId}/messages
{content, clientMessageId, parentId?}
JWT auth middleware validates the token first
(401 UNAUTHORIZED otherwise).
sendMessage(userId, channelId,
content, clientMessageId, parentId)
canView(userId, channelId)
allowed / denied
alt
[channel missing or not viewable]
404 CHANNEL_NOT_FOUND
[channel is viewable]
findByClientMessageId(userId, clientMessageId)
existing message or none
alt
[same clientMessageId already used (duplicate)]
200 OK, the existing message (nothing is created)
[new message]
alt
[invalid content or clientMessageId, or invalid parent]
400 VALIDATION_ERROR / 404 MESSAGE_NOT_FOUND / 409
PARENT_MESSAGE_DELETED
[valid]
Message, mentions and notifications are saved in one transaction. Events
are sent only after it is committed.
save message (createdAt, id)
message saved
saveMentions(message): @tag, @channel (Owner / Admin only)
mentions
createNotifications(message, mentions)
One notification per user: MENTION >
CHANNEL_MENTION > REPLY, never for the author.
Mobile users who are offline also get a push
(FCM / APNs).
publish message.created {channelId, message, clientMessageId} to channel-{channelId}
message.created
adds the message;
unread badge goes
up if the channel
is not open
201 Created, the message (Location:
/api/v1/messages/{messageId})
The author also receives message.created and
matches it with the pending message by
clientMessageId.

### 2.6.2 User flow diagram - Message editing

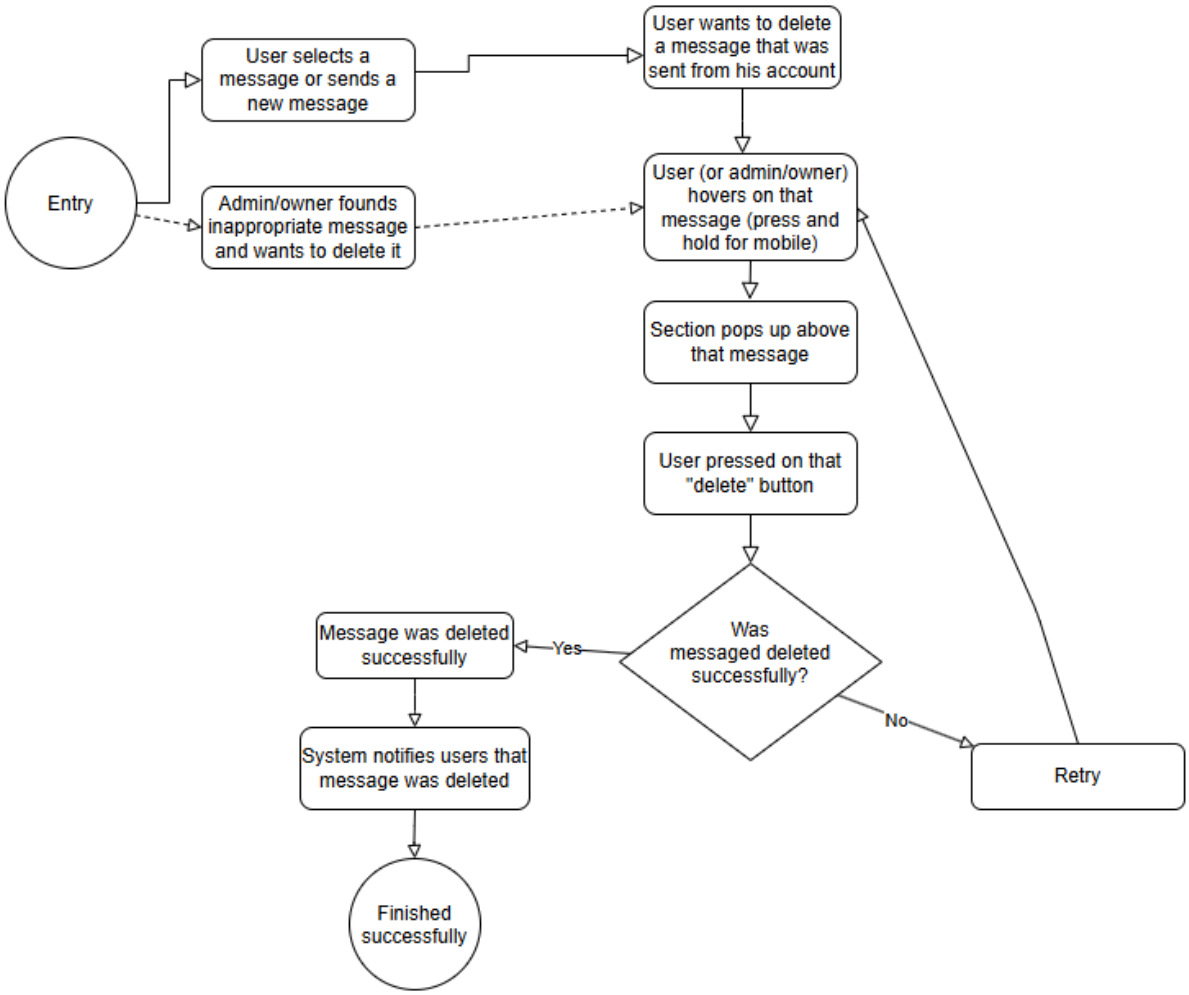
This user flow outlines the end-to-end process for editing a chat message across desktop and mobile interfaces. The flow begins when a user selects an existing message or sends a new one in the chat interface. To trigger the editing options, the user hovers over the target message on desktop or performs a long-press gesture on mobile. This action reveals an overlay menu positioned above the message, from which the user selects the "Edit" option. Once in editing mode, the system checks whether the message content has actually been modified compared to the original version. If no changes are detected, the system requires the user to update the text before allowing submission. When a modified message is submitted, it updates successfully and notifies all users that are in this channel that the message was edited. From there, the user has the choice to either complete the process or attempt another edit. If they choose to edit the message again, the system verifies whether an edit count limit has been reached for that specific message. Reaching the limit completes the flow automatically, while remaining within the limit redirects the user back to the edit interface to make further adjustments. It is not possible to edit a system, other person's or a message that is deleted.




### 2.6.3 User flow diagram - Deleting a message

This user flow applies to every role: Owner, Admin, and Ordinary member. A user can delete only their own messages, meanwhile the owner or an admin can delete their own or other user's messages. When the user hovers over a message they sent, a "Delete" button appears above it. For Members, the button is not shown on other people's messages. After the user presses "Delete", the system checks whether the deletion succeeded. On success, it notifies all users that are in this channel that

the message was deleted and the flow ends. On failure, the user can retry.



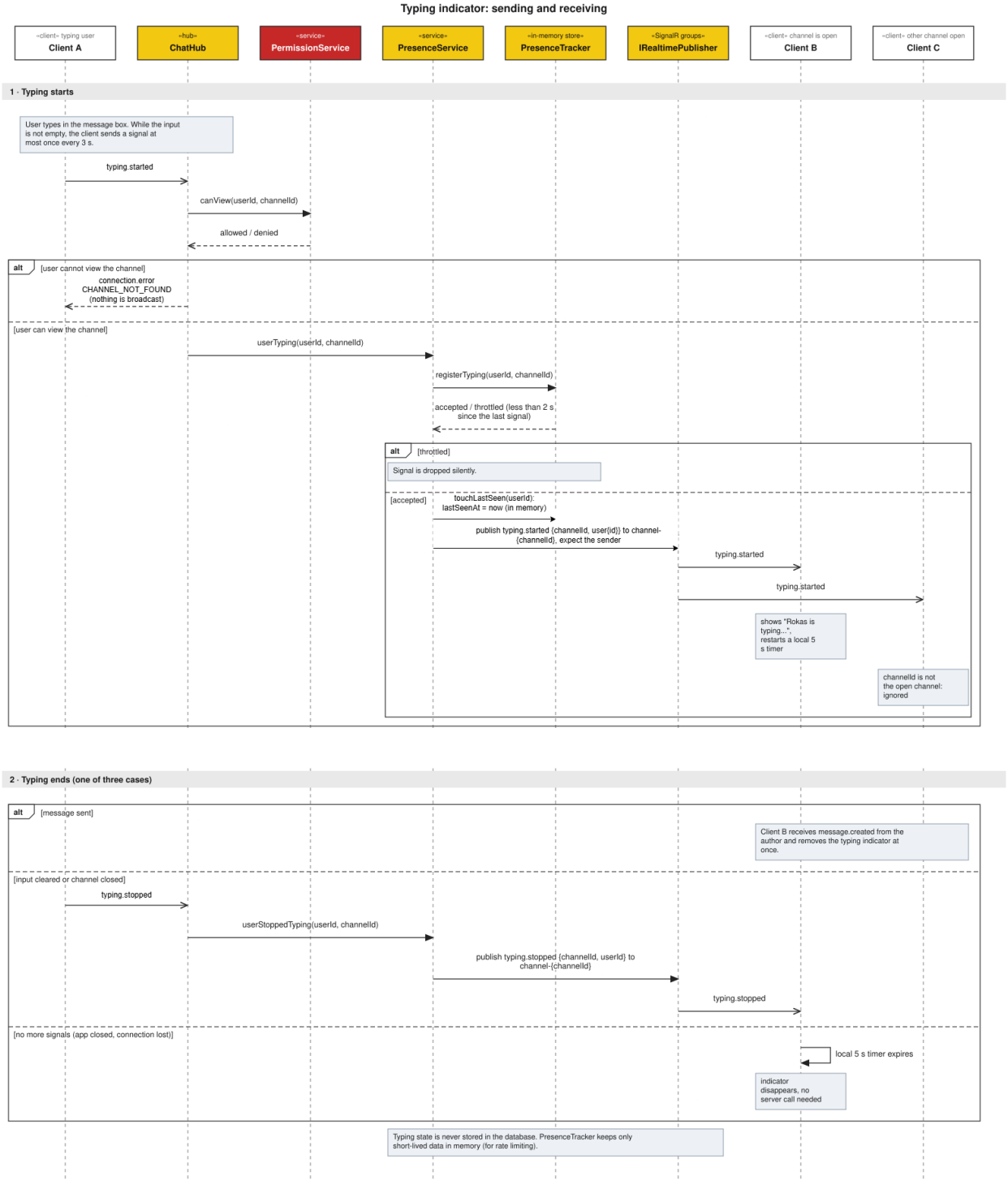

## 2.7 Presence and typing indicators

### 2.7.1 User typing - Sequence diagram

Typing indicators are short-lived signals and are never stored in the database. While the user is typing and the message box is not empty, the client sends a `Typing(channelId)` signal to `ChatHub`, at most once every 3 s. `ChatHub` asks `PermissionService` whether the user can view the channel. If not, it returns `CHANNEL_NOT_FOUND` (a private channel is not revealed) and nothing is broadcast. Otherwise `ChatHub` passes the signal to `PresenceService`, which uses `PresenceTracker` to drop any signal that arrives less than 2 s after the previous one from the same user in the same channel. An accepted signal is published through `IRealtimePublisher` as `typing.started` (with the `channelId` and the user's id, tag and displayName) to the `channel-{channelId}` group, except for the sender's own connections. Every connection belongs to the group of every channel its user can view, which is also how unread badges work, so the event also reaches clients that have another channel open. Each client therefore compares the `channelId` with the channel it has open and ignores events for other channels. Private channels stay private, because their group contains only the users who can view them. A client that receives the event shows "<displayName> is typing..." and starts a local 5 s timer, which restarts with every new signal. The indicator disappears in one of three ways:

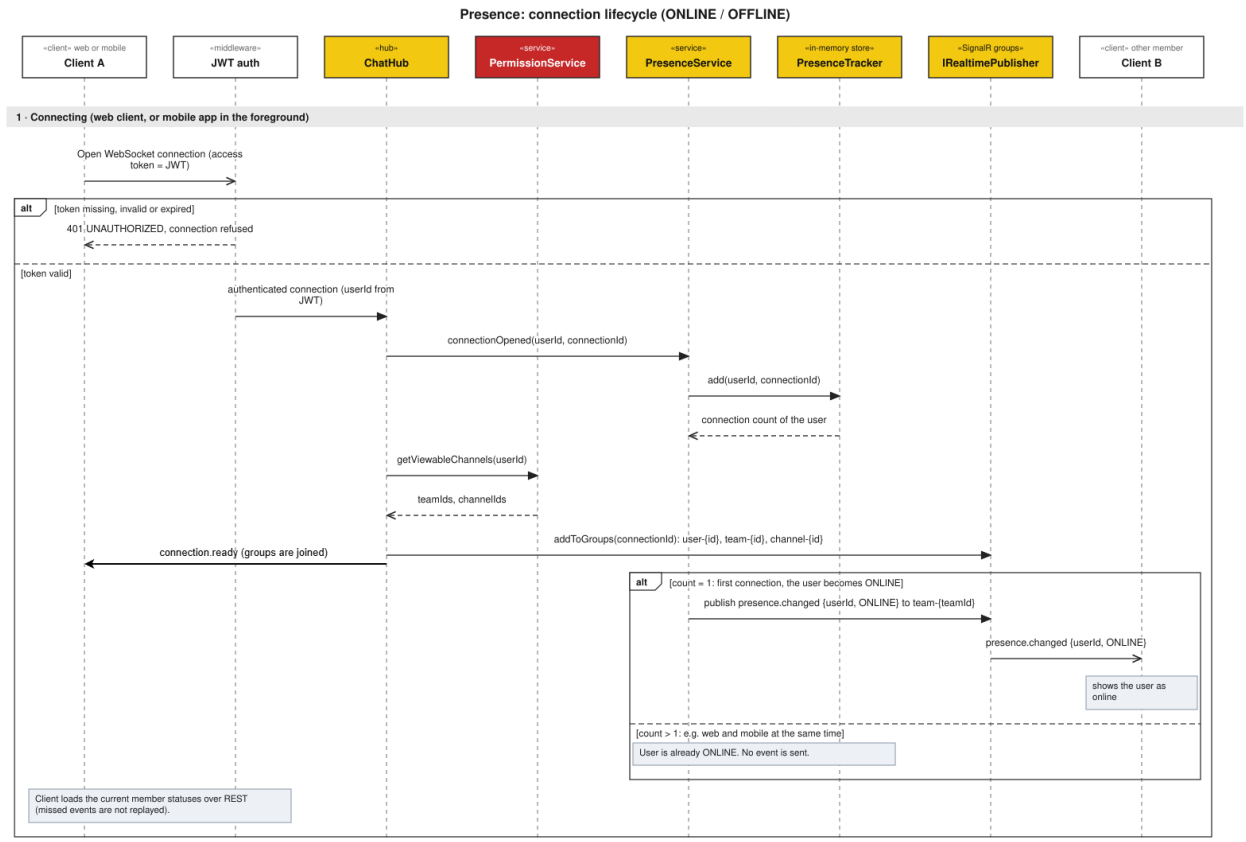
- **Message sent:** the client removes the indicator as soon as it receives `message.created` from that author.

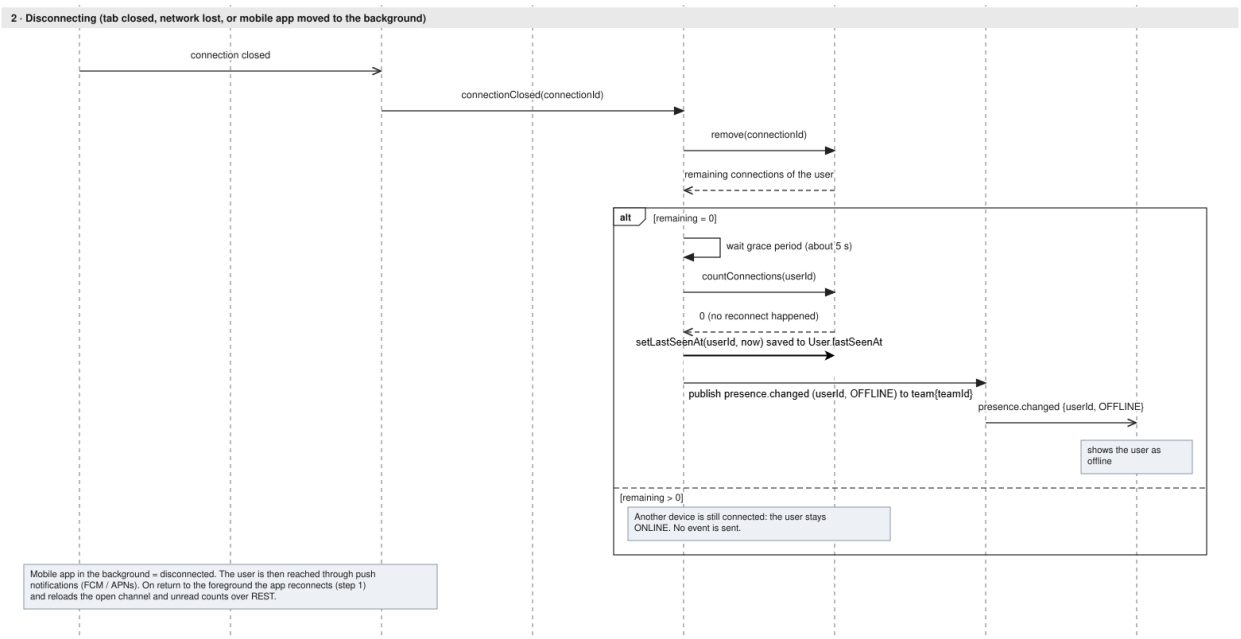
- **Input cleared or channel closed:** the client sends `StopTyping(channelId)`, and the server publishes `typing.stopped`.
- **No further signal:** the local timer expires, for example when the app is closed or the connection is lost, and the indicator disappears without any server call.



### 2.7.2 User presence - Sequence diagram

The system doesn't ask users whether they are online. It infers it from open connections. `PresenceTracker` keeps an in-memory registry that maps each user ID and tag to the set of that user's open connection IDs, and a user is ONLINE if and only if this set is not empty. The registry is not stored in the database, because it changes constantly and a server restart clears it anyway. When a client opens the WebSocket connection with its JWT, the JWT middleware validates the token, and `ChatHub` registers the new connection in `PresenceTracker`. `ChatHub` then asks `PermissionService` which teams and channels the user can see and adds the connection to the matching groups (`user-{userId}`, `team-{teamId}` and `channel-{channelId}`). A user becomes "ONLINE" only when their first connection is opened, at which point `presence.changed` is sent to the team. Opening a second connection, for example the web client alongside the mobile app, sends no event, because the user is already "ONLINE". A client that has just connected has missed earlier presence events, since they are not replayed, so it loads the current statuses once over REST and then applies only the live `presence.changed` events. The process works in reverse when a connection closes. A user becomes "OFFLINE" only when their last connection is gone, and a short grace period before the "OFFLINE" event keeps the status from flickering when a client reconnects quickly. A connection can also vanish without a clean close, for example when a laptop loses its network connection or a phone runs out of battery. The server detects this through keep-alive pings and removes a connection that has been silent for too long (in WebSocket, the server pings every 15 s by default and drops a connection after 30 s of silence). A mobile app that moves to the background closes its connection and therefore counts as disconnected. While the app is in the background, the user is reached through push notifications. When the app returns to the foreground, it reconnects and reloads the open channel and the unread counts over REST.






## 2.8 System messages (join, leave, promote, task updates)

A system message is a message of type `SYSTEM` that the server posts by itself when something happens in the team. It has no author (`author` is null), appears in the timeline like a normal message, and cannot be edited or deleted by anyone. The text is built when the message is created and uses the user's tag, not the display name, so a later rename does not make it outdated. Users cannot send system messages.

System messages are posted in the same operation that causes them. If the operation fails, no message is posted. Each one is delivered live as `message.created` with an empty author (see WebSocket - Messaging, presence and typing events).

### 2.8.1 When system messages appear

| Event | Text | Channel | Posted when |
|---|---|---|---|
| Join | @<tag> joined the team | `#general` | A user joined the team with an invite code |
| Leave | @<tag> left the team | `#general` | A user left the team on their own (Admin, Member) |
| Removed | @<tag> was removed from the team | `#general` | The Owner removes a member from the team |
| Promote | @<tag> was promoted to Admin | `#general` | The Owner changes a Member's role to Admin |
| Demote | @<tag> was demoted to Member | `#general` | The Owner changes an Admin's role to Member |
| Transfer | @<tag> is now the Owner of the team | `#general` | The Owner transfers the team to an Admin, or an inactive Owner's ownership is claimed (see 2.3) |
| Task created | @<tag> created the task "<title>" | `#general` | A task is created |

| Event | Text | Channel | Posted when |
|---|---|---|---|
| Task assigned | @<tag> assigned the task “<title>” to “@<assignedTag>” | `#general` | An Owner or Admin assigns a task, or reassigns it to another person |
| Task self-assigned | @<tag> took the task “<title>” | `#general` | A user self-assigned an open task |
| Task unassigned | @<tag> unassigned the task “<title>” | `#general` | A task is unassigned |
| Task status changed | @<tag> moved the task “<title>” to <status> (for example “to In progress”) | `#general` | Status changes to OPEN, IN_PROGRESS or PENDING_REVIEW |
| Task approved | @<tag> approved the task “<title>” | `#general` | PENDING_REVIEW becomes DONE |
| Task returned | @<tag> returned the task “<title>” to “In progress” | `#general` | PENDING_REVIEW goes back to IN_PROGRESS with a comment |

### 2.8.2 Rules

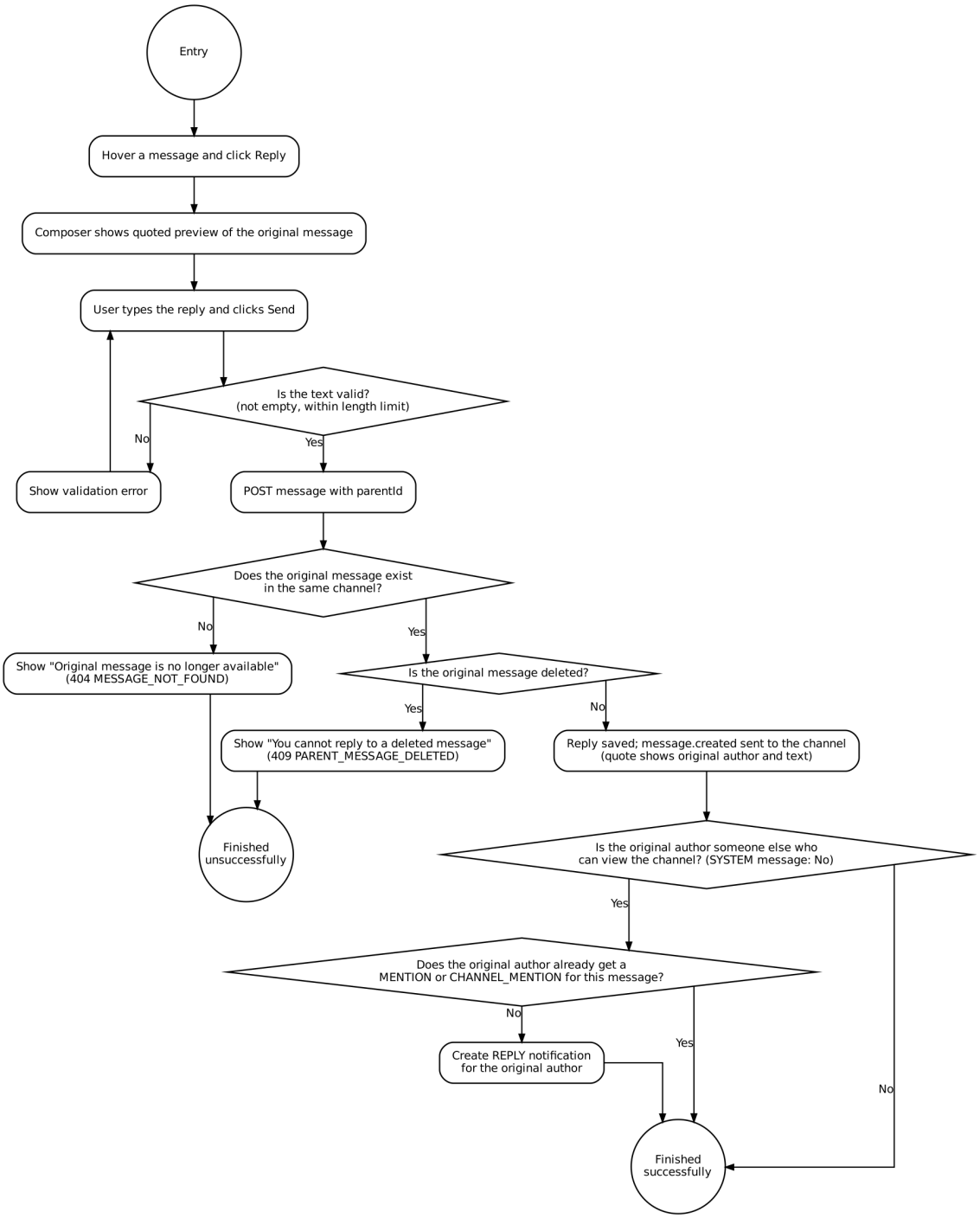
| Rule | Value |
|---|---|
| Channel | Always `#general`, also when the user was looking at another channel. The leave message does not go to the “active channel”, because the user who leaves has no active channel afterwards |
| Pins | Pinning or unpinning a message does not create a system message |
| Notifications | System messages create no Notification records. They are not mentions or replies, they do not show in the notification list, and the notifications of task changes are created separately by the task flow (see 2.4) |
| Unread | A system message counts in the unread badge of `#general` like any other message. It has no author, so it counts for every user, including the one whose action caused it |
| Reply | Anyone who can view `#general` can reply to a system message (see 2.9) |
| Edit, delete | Not possible for anyone, `403 FORBIDDEN` |
| Removed or leaving user | The message is posted after the membership is removed, so the user does not receive it |
| Search | System messages are not returned by search (see 2.13) |

## 2.9 Replies

A reply is a normal channel message that points to an earlier message in the same channel through `parentId`. It is shown in the timeline with a quote of the original (author and preview text). Threads are Won’t have, so replies stay in the main timeline in chronological order. Error codes are listed in REST - Replies, reactions, pins, notifications, search.

| Rule | Value |
|---|---|
| Who / what | Anyone who can view the channel may reply to any non-deleted USER or SYSTEM message of the same channel, also to a reply (no nesting). Content rules as for a normal message. `parentId` cannot be changed later |
| Quote | Author's displayName (none for SYSTEM) and the preview text, always the current text of the original. If the original is deleted later, the reply stays and the quote shows "Message deleted" |
| Notification | REPLY to the original author, unless they are the replier, can no longer view the channel, or already get a MENTION or CHANNEL_MENTION for the same message |
| Clicking the quote | Opens the timeline around the original (`GET messages?around=`) and highlights it for 2 s |

**User flow diagram - Replying to a message**




## 2.10 Reactions and pinned messages

Reactions: an emoji chip with a counter under the message; hovering (on mobile, a long press on the reaction chip) shows who reacted. Clicking toggles the user's reaction: the client calls POST when `reactedByMe` is false and DELETE when true (the server never toggles). Pins: Owner / Admin pin messages; the newest pin is shown in a banner above the timeline and all pins in the Pinned (n) list. Both are single-step actions, so they have no flow diagram.

| Rule | Value |
|---|---|
| Who | React: anyone who can view the channel. Pin / unpin: Owner, Admin (not shown to Members) |
| Emoji value | One Unicode grapheme cluster starting with an `Extended_Pictographic` or `Regional_Indicator` code point, or a keycap sequence; max 32 code points. Normalised to NFC with U+FE0F added where needed (the heart symbol with and without U+FE0F is the same reaction) |
| Uniqueness | One reaction per user, emoji and message; a user may add several different emoji |
| Limits | Max 20 different emoji per message; max 50 pins per channel. Checked after locking the message row (reactions) or channel row (pins) with `SELECT ... FOR UPDATE` |
| Deleted message | Reactions hidden (empty list), React and Pin not shown; a pinned message that is deleted is unpinned automatically (`message.unpinned`, `unpinnedBy` = null) |
| Pinned list/banner | Newest pin first (`pinnedAt`, then `id` descending); the banner shows the first item, clicking it opens the pinned list |
| Notifications, system messages | None: pinning and unpinning create no notification and no system message |

### 2.11 Mentions and notifications

Typing `@` in the composer opens a suggestion list; choosing a person inserts their `@tag`. The server (not the client) finds the mentions when the message is sent or edited, saves them, and creates notifications. The bell shows unread notifications and opens the list.

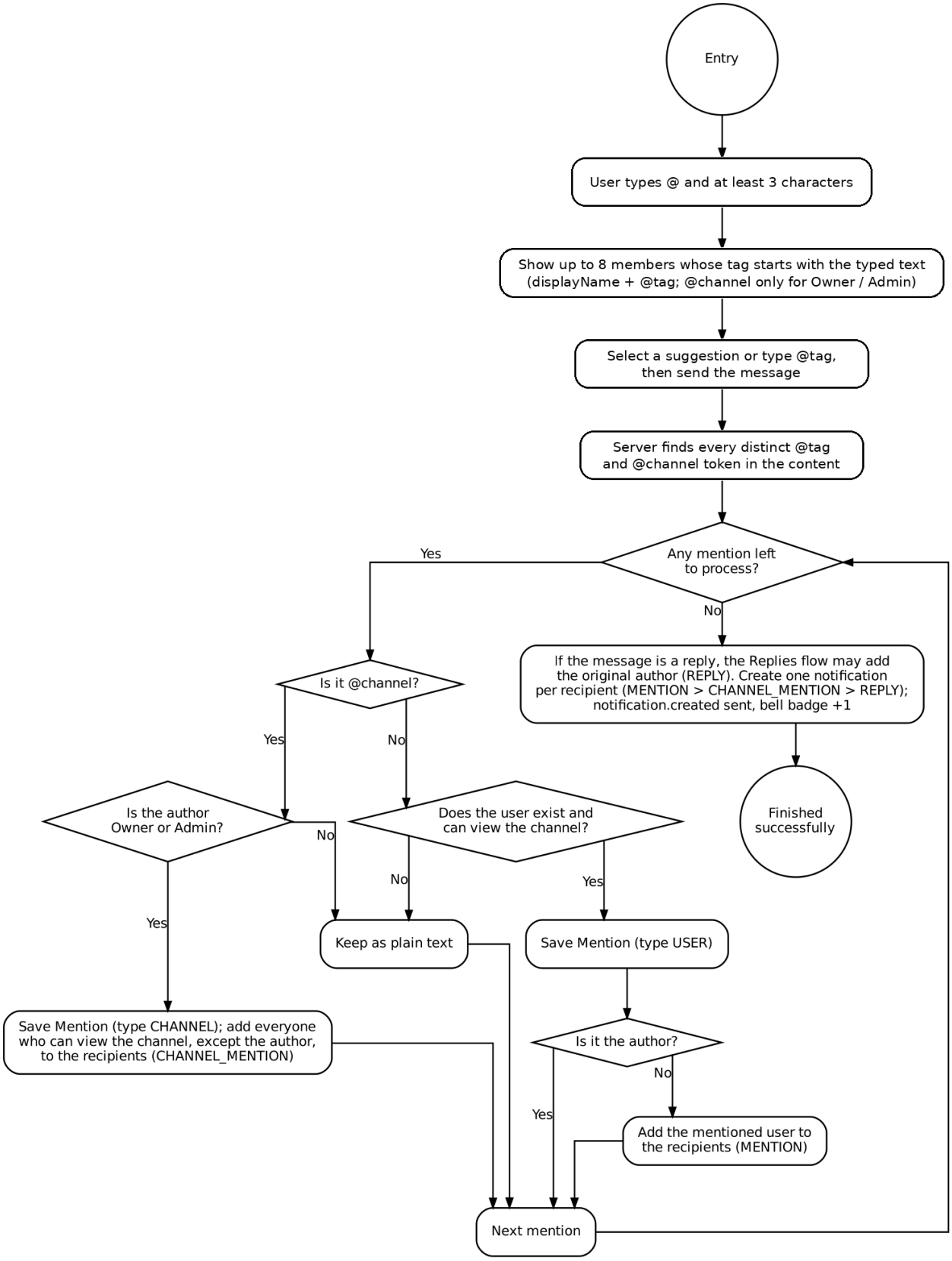
| Rule | Value |
|---|---|
| Syntax | `@` at the start of the text or after a space/line break, then 3-32 characters `[A-Za-z0-9_.]`; the token ends at the first other character, and a period at the end of the token is not part of it (so "@rokas." mentions rokas). Matched to the tag ignoring case. `@channel` is the channel mention ("channel" is a reserved tag) |
| Who can be mentioned | Users who can view the channel (all team members for public channels, channel members for private ones). Others, and unknown names, stay plain text with no Mention row |
| Suggestions | Members whose tag starts with the typed text, shown with their displayName; the list opens after at least 3 characters are typed, max 8. `@channel` only offered to Owner / Admin |
| `@channel` | Owner / Admin only (checked again on edit); one Mention row (type CHANNEL); notifies everyone who can view the channel except the author. From a Member it stays plain text and the message is still sent (deliberate exception to the 403 rule) |
| Self and repeats | Mentioning yourself stores a Mention but no notification; a user mentioned several times gets one Mention and one notification |
| Edits | Mentions are parsed again; removed ones are deleted. Only users never notified about this message get a new notification |

| Rule | Value |
|---|---|
| Rendering | The client does not search the text: it highlights the ranges listed in the message's `mentions` array and shows the mentioned user's current @tag there, so mentions stay correct after a tag change. The current user's mentions and `@channel` are highlighted more strongly |

| Notification type | Created when |
|---|---|
| `MENTION` / `CHANNEL_MENTION` | The user is mentioned with `@tag` / Owner or Admin uses `@channel` in a channel the user can view |
| `REPLY` | Someone replies to the user's message (see Replies) |
| `TASK_ASSIGNED` | A task is assigned to the user by someone else (also reassigning). Taking an open task yourself creates no notification, because the actor never notifies themselves |
| `TASK_STATUS_CHANGED` | A task the user created or is assigned to changes status, made by someone else |

| Rule | Value |
|---|---|
| One per message | A user gets at most one notification per message; priority MENTION > CHANNEL_MENTION > REPLY. The actor never notifies themselves |
| Preview | Message types: the preview text; task types: the task title |
| Badge and list | Bell shows unread visible notifications of the current team (9+ above 9). Opening one marks it read and opens the channel around the message or the task board with the task. "Mark all as read" marks the whole team |
| Deleted/lost access | Deleted message: the notification stays and opens "Message deleted". Lost access to the channel or task: the notification is hidden (not listed, not counted, PATCH returns 404). Leaving or being removed from the team deletes the user's notifications and read states of that team |

### User flow diagram - Mentioning members

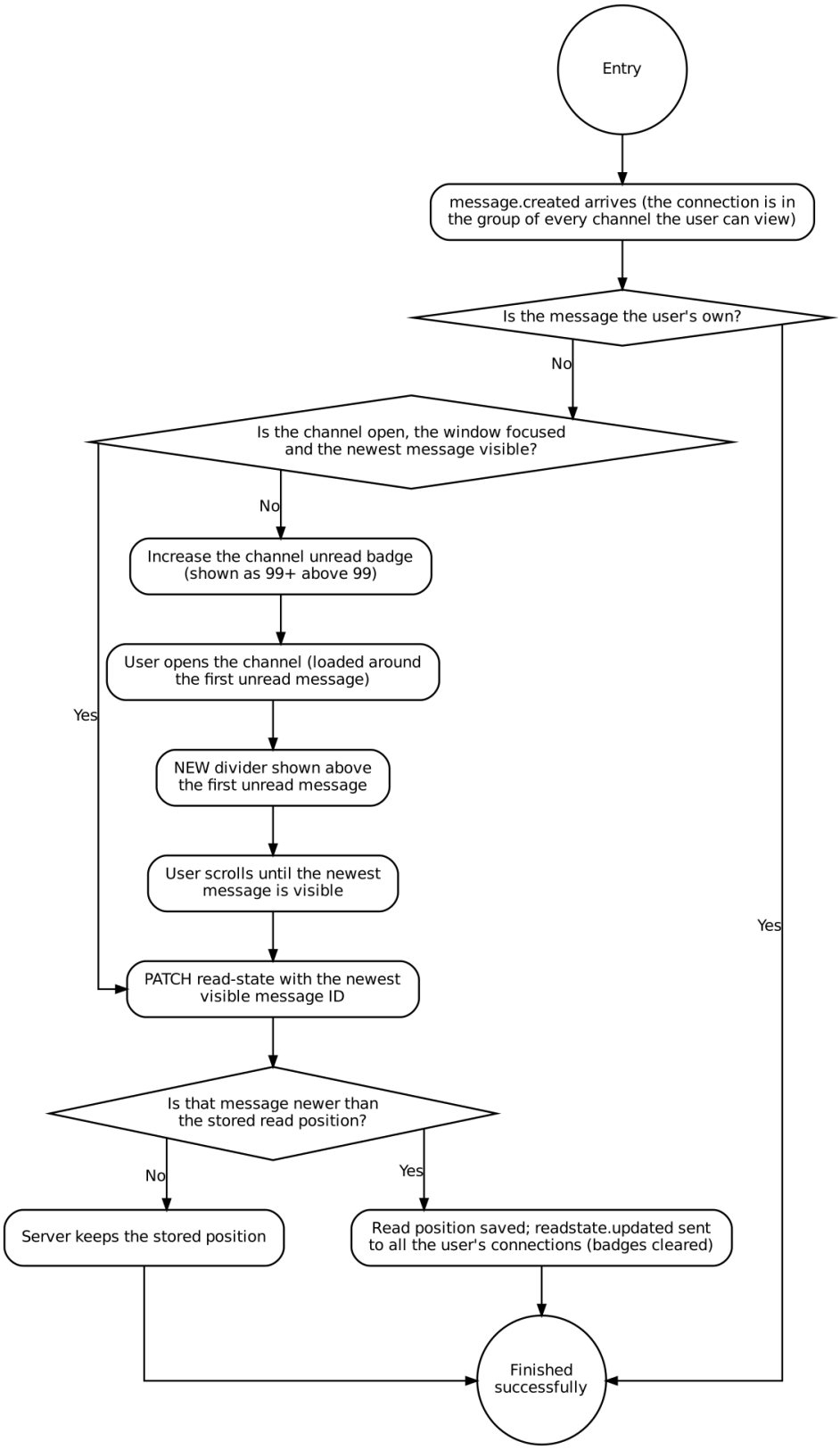


### 2.12 Unread indicators and message search

For every channel the user can view, `ReadState` stores how far the user has read. Channels with new messages show a red badge (99+ above 99, none when 0), and opening one shows a NEW divider above the first unread message, which disappears when the user leaves the channel. Badges update live because every connection is in the group of every channel the user can view. Search (Should have) is a single request and response, so it has no flow diagram.

| Rule | Value |
|---|---|
| Order | Messages are ordered by `(createdAt, id)`, never by `id` alone; with no `lastReadMessageId` compare by `createdAt > lastReadAt` |
| ReadState lifecycle | Created when the user gets access (joins the team, a channel is created, added to a private channel) pointing to the newest message (null if empty), `lastReadAt` = its `createdAt` or the access time. Deleted when access is lost; regaining access creates a new row |
| Unread | Messages after `(lastReadAt, lastReadMessageId)`, not deleted, and not written by the user: `authorId IS NULL OR authorId <> userId` (system messages count) |
| Marking as read | When the channel is open, the window focused and the newest message visible, the client sends that message's ID. The position never moves backwards; a deleted message is accepted. Own messages never change the badge |
| NEW divider | On opening a channel with unread messages the client loads `GET messages?around=<firstUnreadMessageId>` and shows the divider above it |
| Several devices | `readstate.updated` goes to all the user's connections, so badges clear everywhere |
| Search | One team, optionally one channel; query 2-100 characters, case-insensitive substring (% and _ literal); only channels the user can view; no deleted or system messages; newest first; result shows channel, author, time and highlighted text; opening it shows the channel around the message, highlighted for 2 s |

**User flow diagram - Unread indicators**



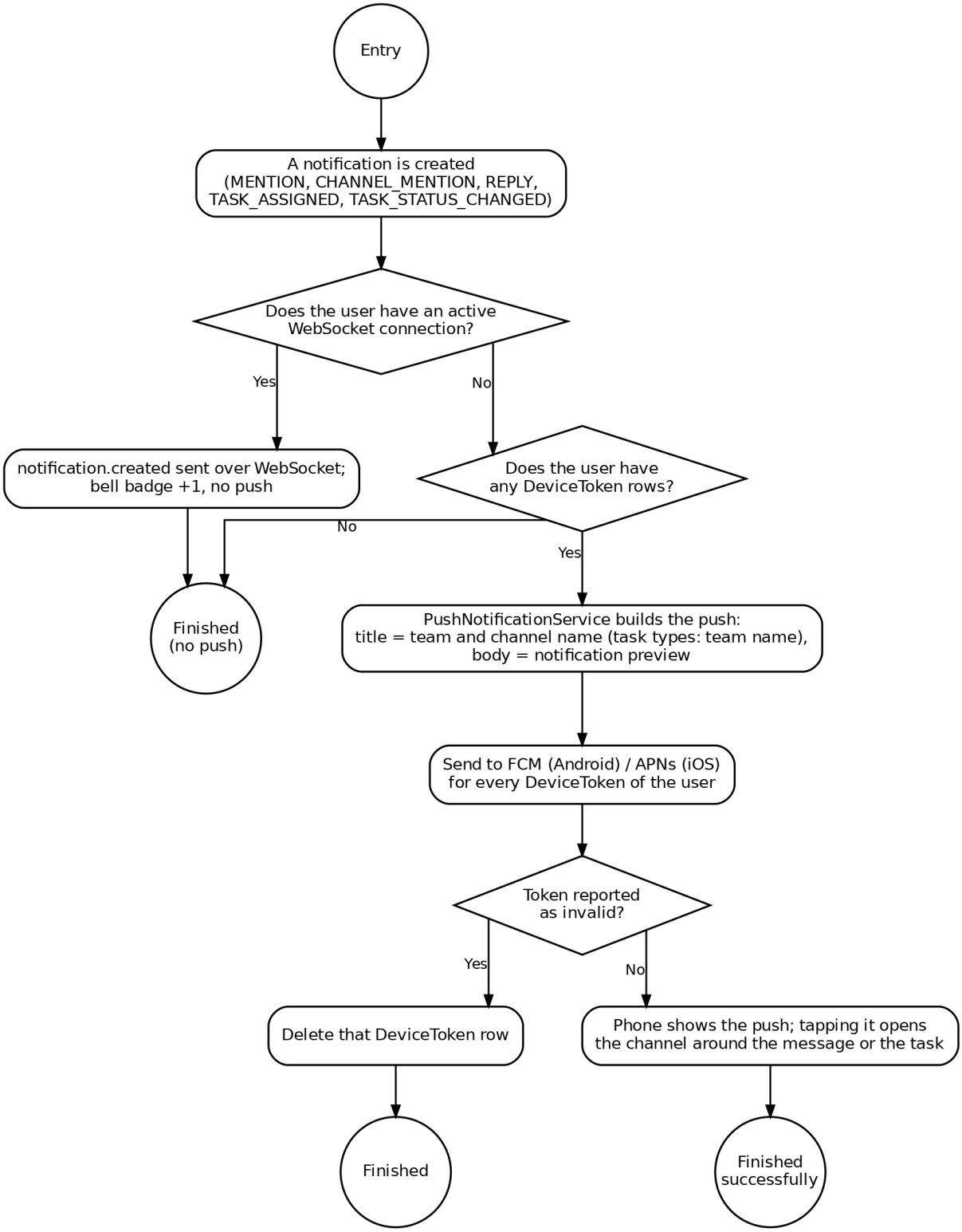

## 2.13 Push notifications

Push notifications reach a user whose clients are all disconnected, for example when the mobile app is closed or in the background. Every push is created from a Notification row (see Mentions and

notifications), so the bell and the push always show the same events. The server sends them through `PushNotificationService` to FCM (Android) and APNs (iOS), using the user's `DeviceToken` rows.

| **Rule** | **Value** |
|---|---|
| When | A notification is created and the user has no active WebSocket connection (web, or mobile in the foreground). A user with the web client open gets no push, even if the app on the phone is closed |
| Types sent as push | MENTION, CHANNEL_MENTION, REPLY, TASK_ASSIGNED, TASK_STATUS_CHANGED; ordinary messages are not pushed |
| Content | Title: team and channel name for message types, team name for task types; body: the notification preview (message preview text or task title). Sent only to users who can view the channel or task |
| Devices | Sent to every `DeviceToken` of the user |
| Tapping it | Opens the app on the message (channel around the message) or on the task |
| Invalid token | If FCM/APNs reports the token as invalid, the server deletes the `DeviceToken` row |

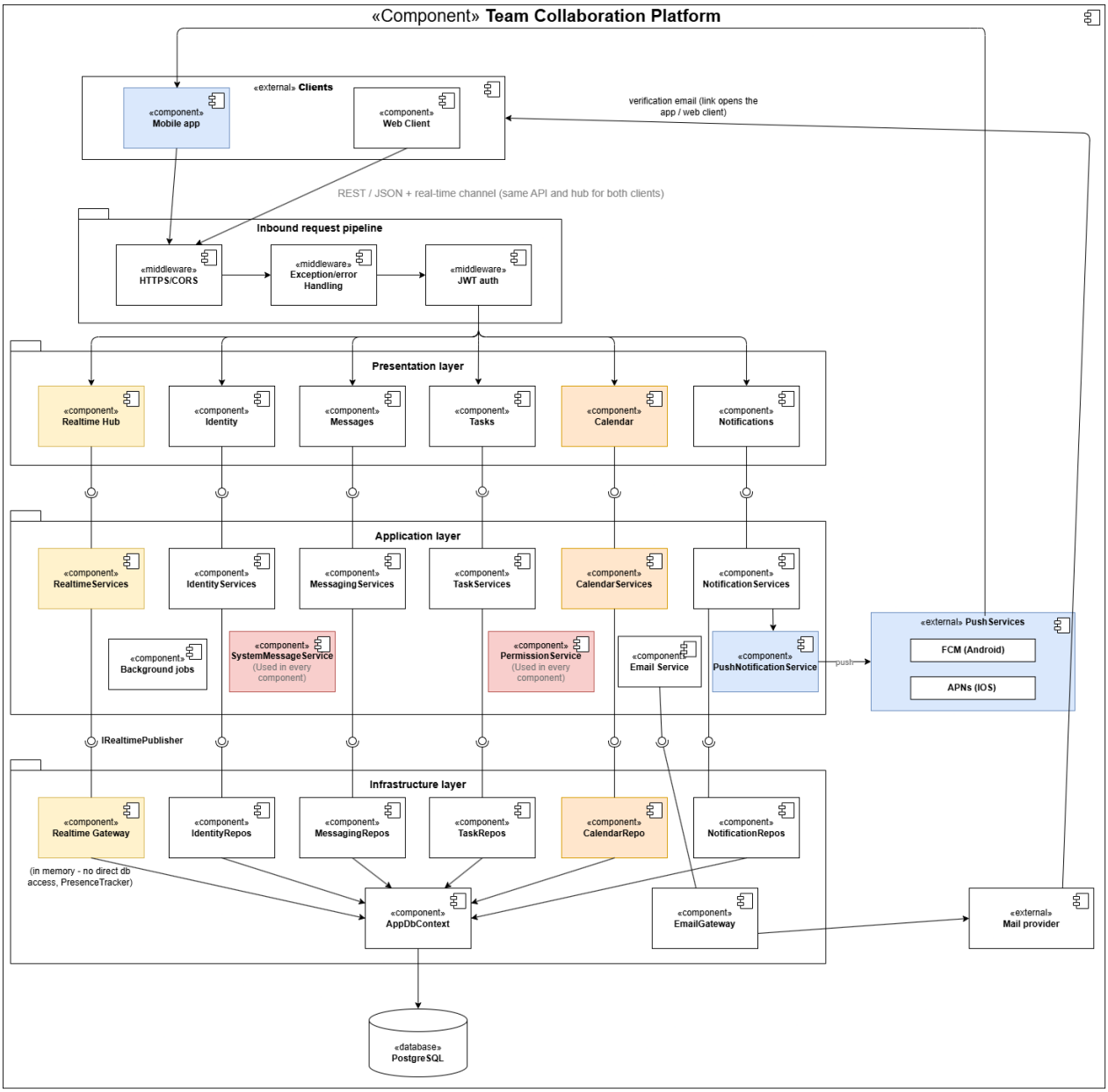
### User flow diagram - Push notifications



# 3 Architecture

## 3.1 Package diagram (high-level architecture)

This diagram, a combination of component and package diagrams, presents our team's vision for implementing the system. As noted in the technology stack section (3.2), it is a reference architecture, not a mandatory one. Other teams are free to choose their own technologies and implementation approach. The real-time layer is optional: the WebSocket could be replaced or removed, and the client then works through REST only, without live updates, typing indicators and online status. Every layer is explained below:




**Clients (external).** The web client and the native mobile app. Both talk to the same REST API and the same real-time hub. They contain no business logic of their own, only the UI and the calls to the backend.

**Inbound request pipeline.** The middleware every request passes through before reaching the controllers: HTTPS/CORS, error handling (turns exceptions into the standard error format and status codes), and JWT auth (checks the token and identifies the user).

**Presentation layer.** The entry points of the system: controllers for Identity, Messaging, Tasks, Calendar and Notifications, plus the Realtime Hub. They receive requests, validate the input, and pass the work to the matching service. They contain no business rules.

**Application layer.** The business logic, grouped by module: identity and teams, messaging (channels, messages, reactions, pins, mentions, search), tasks, calendar, notifications and read state, and presence. Two shared services serve every module: `PermissionService` (role and visibility checks) and `SystemMessageService` (automatic messages such as join, leave, promote and task updates). `PushNotificationService` sends push messages to the external push services, and `EmailService` sends the verification e-mail through the external mail provider. `BackgroundJobs` run on a timer, without a user request: they delete invites 7 days after they expire or are revoked, and remove accounts that were not verified in time. They use the Identity services and repositories.

**Infrastructure layer.** Everything that talks to storage and technical systems: the repositories of each module, `AppDbContext` (the single access point to the database) and the Realtime Gateway, which sends events to connected clients and tracks who is online. Only this layer knows how data is stored.

**External systems.** PostgreSQL stores all data. FCM (Android) and APNs (iOS) deliver push notifications to phones where the app is closed or in the background. Mail providers used to verify every user through email.

Colors are used as follows:

- **Yellow:** WebSocket (real-time) packages and components. A team that chooses a different approach can omit them without affecting the rest of the architecture.
- **Orange:** components outside the Must-have scope, such as the personal Calendar. Our scope definition classifies it as Could-have.
- **Red:** `SystemMessageService` and `PermissionService`. Every module in the Application layer uses them. They are shown once, highlighted, instead of being repeated in each module.
- **Blue:** This color was used to represent every package/module/component that is related to the mobile app. That was also mentioned in our scope definition as Could have.

### 3.2 Technology stack

Defining the technology stack is outside the scope of this project, and as a team we do not wish to restrict or limit the choices of other teams that will continue working on this project. However, if we were to implement this application ourselves, we would use the following stack:

| **Component** | **Technology** |
|---|---|
| Backend | ASP.NET Core Web API |
| Websocket (for real-time communication) | SignalR |
| Frontend | React, TypeScript, Vite, Tailwind CSS |
| Database | PostgreSQL |
| ORM | Entity Framework Core with Migrations |
| Auth | ASP.NET Core Identity with JWT Bearer tokens |
| Mobile application | React Native (Expo) with TypeScript and push via Firebase Cloud Messaging (for Android), Apple Push Notification service (for iOS) |

This reflects our own preference only. The following alternatives are equally suitable:

| Component | Alternatives |
|---|---|
| Backend with websockets | NestJS (Node.js/TypeScript) with Socket.IO; Spring Boot with STOMP/WebSocket; FastAPI or Django with Channels; Go (Fiber/Gin) with gorilla/websocket |
| Frontend | Next.js (React); Vue 3 with Vite and Pinia; SvelteKit; Angular |
| Database | MySQL/MariaDB; Microsoft SQL Server. Redis can be added for caching, presence tracking and pub/sub |
| ORM | Prisma, Drizzle or TypeORM (Node.js); Hibernate/JPA (Java); SQLAlchemy with Alembic (Python); Django ORM |
| Auth | Auth.js; Passport.js with JWT; Keycloak; Firebase Auth or Supabase Auth; Spring Security |
| Mobile application | Flutter, .NET MAUI, native Kotlin/Swift. |

The WebSocket layer is optional: without it the client works through REST only, without live updates, typing indicators and online status. Other teams may choose an alternative real-time mechanism, such as:

- Server-Sent Events (SSE)
- Managed real-time platforms (PaaS)
- WebTransport
- HTTP long polling

Note that the spec says the client sends only typing signals over the WebSocket (presence is inferred from open connections), and the server pushes all other events. This is why SSE or long polling can replace the WebSocket for everything except typing: both are one-way (server to client). A team that chooses them must add its own REST endpoint for typing signals (not defined in this specification) or drop typing indicators. Presence needs a persistent connection, because the server infers online status from open connections; with long polling it would have to be defined differently, for example with a heartbeat endpoint, which this specification does not define.

# 4 LoFi wireframes

The low-fidelity wireframes for both clients are maintained in Figma rather than embedded in this document, so they stay up to date and can be viewed at full size.

**Wireframes:** View Figma Wireframes

The Lo-fi wireframes page contains every screen of the web application and the mobile app, grouped by feature. Each group shows the web screens on the left and the mobile screens on the right. Numbered markers on a screen refer to the notes below it.

**Clickable prototype:** View Figma Prototype

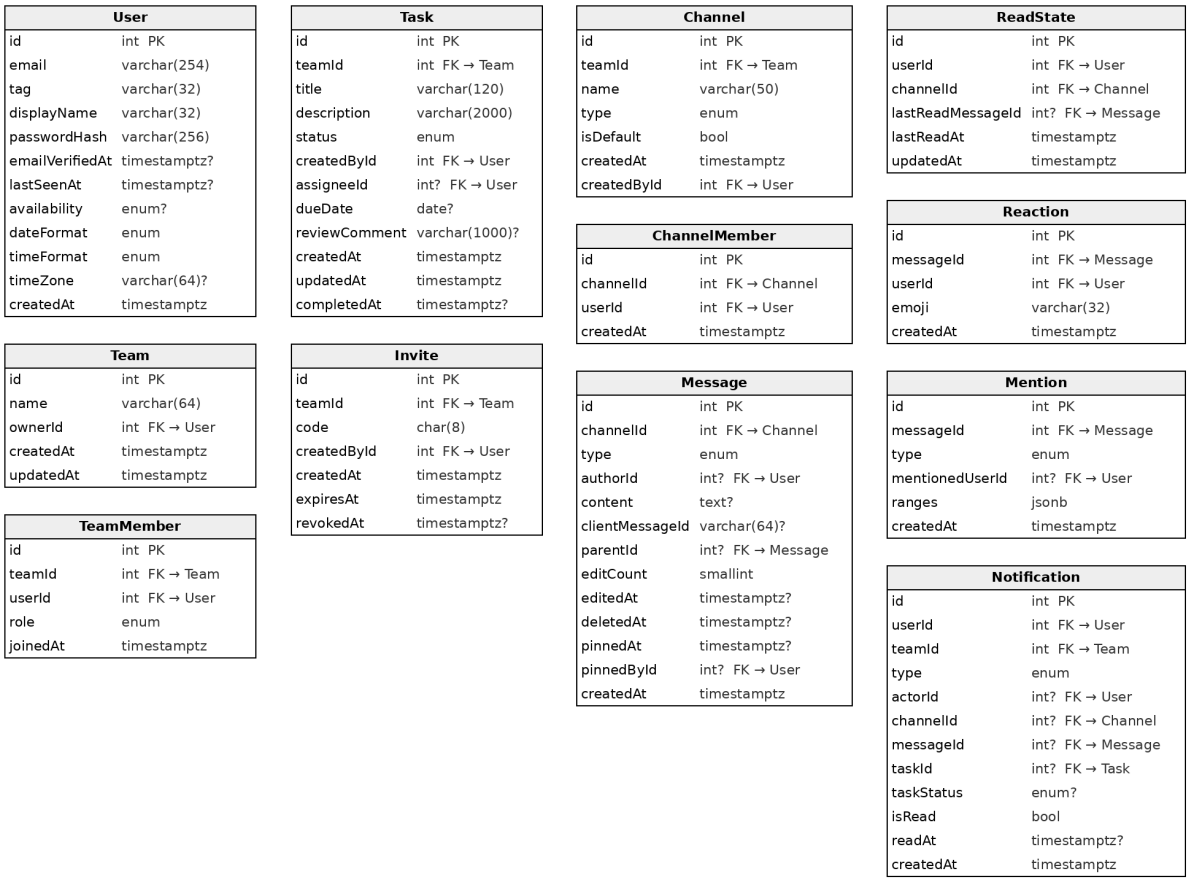
The same screens are linked into two flows, Web app and Mobile app, starting from the login screen. Click buttons, tabs and list items to move between screens. To see every screen, open the flow list with the sidebar icon in the top-left corner and choose Web - all screens or Mobile - all screens. Each starts with a clickable list of all screens.

All screens use the same sample team (Vilkai) and members. Controls marked Owner/Admin are hidden from Members. Team settings are web only, so the mobile app shows a notice instead.

# 5 Data model

## 5.1 Complete data model diagram

All business entities with fields and data types (PostgreSQL). "FK → X" is a foreign key to `X.id`, "?" means nullable. On conflict, the table sections below win. Common rules: int auto-increment ids; timestamps are `timestamptz` (C# `DateTime`, `Kind = Utc`), truncated to milliseconds; enums stored as UPPER_SNAKE_CASE strings; names in camelCase, as in the API.




PK = primary key, FK → X = foreign key to X.id, ? = nullable.

Authentication tables (specified in User, team, TeamMember, Invite, Task):

| EmailVerificationToken | |
|---|---|
| id | int PK |
| userId | int FK → User |
| tokenHash | varchar(128) |
| expiresAt | timestamptz |
| usedAt | timestamptz? |
| createdAt | timestamptz |

| RefreshToken | |
|---|---|
| id | int PK |
| userId | int FK → User |
| tokenHash | varchar(128) |
| expiresAt | timestamptz |
| revokedAt | timestamptz? |
| createdAt | timestamptz |

| DeviceToken | |
|---|---|
| id | int PK |
| userId | int FK → User |
| platform | enum |
| token | varchar(512) |
| createdAt | timestamptz |
| lastSeenAt | timestamptz |

## 5.2 User, team, TeamMember, Invite, Task

### User

A registered account. Users who have joined a team are never deleted, so messages and tasks always point to an existing user. Only unverified accounts are deleted after 7 days; they have no teams or messages.

| Field | Type | Constraints / description |
|---|---|---|
| `id` | int | PK, auto-increment |
| `email` | varchar(254) | RFC 5322 address, stored as entered; unique case-insensitively (unique index on `lower(email)`) |
| `tag` | varchar(32) | `[a-z0-9_.]`, 3–32 characters, stored in lowercase, cannot start or end with a period, no two periods in a row; unique; reserved names and blocklist as in Registration and login |
| `displayName` | varchar(32) | 1–32 characters, not unique; defaults to the tag |
| `passwordHash` | varchar(256) | Hash only, never the password |
| `emailVerifiedAt` | timestamptz, nullable | Null until the email is confirmed; login requires a value |
| `lastSeenAt` | timestamptz, nullable | Last login, token refresh or WebSocket connection; used for the inactive Owner rule |
| `availability` | enum, nullable | `AVAILABLE`, `BUSY`, `DO_NOT_DISTURB`; null = not set. Chosen by the user and shown next to the online status |
| `dateFormat` | enum | `AUTO`, `DMY`, `MDY`, `YMD`; default `AUTO` |
| `timeFormat` | enum | `AUTO`, `H24`, `H12`; default `AUTO` |
| `timeZone` | varchar(64), nullable | IANA zone, e.g. `Europe/Vilnius`; null = device time zone |
| `createdAt` | timestamptz | |

### Team

The top-level workspace. On delete: CASCADE to TeamMember, Invite, Task, Channel, Notification.

| Field | Type | Constraints / description |
|---|---|---|
| `id` | int | PK, auto-increment |
| `name` | varchar(64) | 1-64 characters, trimmed; not unique |
| `ownerId` | int | FK User.id; RESTRICT. Always the user whose TeamMember role is `OWNER`; both are updated in the same transaction on transfer |
| `createdAt` | timestamptz | |
| `updatedAt` | timestamptz | |

### TeamMember

One row per user and team; deleted when the user leaves or is removed. On delete: CASCADE from Team, User.

| Field | Type | Constraints / description |
|---|---|---|
| id | int | PK, auto-increment |
| teamId | int | FK Team.id |
| userId | int | FK User.id; unique (teamId, userId) |
| role | enum | OWNER, ADMIN, MEMBER; at most one OWNER per team is enforced by a partial unique index on teamId where role = OWNER; exactly one is guaranteed by the transactions that create the team and transfer ownership |
| joinedAt | timestamptz | |

**Invite**

A multi-use join code. A background job deletes invites 7 days after they expire or are revoked. On delete: CASCADE from Team; createdById RESTRICT (users who joined a team are never deleted).

| Field | Type | Constraints / description |
|---|---|---|
| id | int | PK, auto-increment |
| teamId | int | FK Team.id |
| code | char(8) | Uppercase letters and digits without 0, O, 1, I, L: 31 symbols, so 31^8 ≈ 8.5 × 10^11 possible codes. Generated randomly by the server; if it already exists, a new one is generated. Because old invites are deleted, codes cannot run out; unique |
| createdById | int | FK User.id; never null |
| createdAt | timestamptz | |
| expiresAt | timestamptz | createdAt + 7 days |
| revokedAt | timestamptz, nullable | Set when revoked; the invite stops working at once |

**Task**

A unit of work on the team's board. On delete: CASCADE from Team (and to Notification); createdById RESTRICT (users are never deleted); assigneeId SET NULL.

| Field | Type | Constraints / description |
|---|---|---|
| id | int | PK, auto-increment |
| teamId | int | FK Team.id |
| title | varchar(120) | 1-120 characters |
| description | varchar(2000) | Default empty string |
| status | enum | OPEN, IN_PROGRESS, PENDING_REVIEW, DONE |
| createdById | int | FK User.id; never null |
| assigneeId | int, nullable | FK User.id; must be a team member; null = unassigned |
| dueDate | date, nullable | YYYY-MM-DD, no time |

| Field | Type | Constraints / description |
|---|---|---|
| reviewComment | varchar(1000), nullable | Last comment when returned from review; cleared on DONE |
| createdAt | timestamptz | |
| updatedAt | timestamptz | |
| completedAt | timestamptz, nullable | Set when status becomes DONE |

**EmailVerificationToken**

One row per verification email. On delete: CASCADE from User.

| Field | Type | Constraints / description |
|---|---|---|
| id | int | PK, auto-increment |
| userId | int | FK User.id |
| tokenHash | varchar(128) | Hash of the random token sent in the link; unique |
| expiresAt | timestamptz | createdAt + 24 hours |
| usedAt | timestamptz, nullable | Set when the link is opened; a used token stops working |
| createdAt | timestamptz | |

**RefreshToken**

One row per issued refresh token. On delete: CASCADE from User.

| Field | Type | Constraints / description |
|---|---|---|
| id | int | PK, auto-increment |
| userId | int | FK User.id |
| tokenHash | varchar(128) | Hash of the token; the token itself is never stored; unique |
| expiresAt | timestamptz | createdAt + 30 days |
| revokedAt | timestamptz, nullable | Set on rotation or logout |
| createdAt | timestamptz | |

**DeviceToken**

One row per mobile device for push notifications. On delete: CASCADE from User.

| Field | Type | Constraints / description |
|---|---|---|
| id | int | PK, auto-increment |
| userId | int | FK User.id |
| platform | enum | ANDROID, IOS |
| token | varchar(512) | Push token from FCM/APNs; unique |
| createdAt | timestamptz | |

| Field | Type | Constraints / description |
|---|---|---|
| lastSeenAt | timestamptz | Last time this token was registered or refreshed with POST /devices (informational; invalid tokens are deleted when FCM/APNs reports them, see 2.13) |

**Indexes.** User unique (lower(email)), unique (tag). TeamMember unique (teamId, userId), (userId), partial unique (teamId) where role = OWNER. Invite unique (code), (teamId, expiresAt). Task (teamId, status, updatedAt desc), (assigneeId, dueDate). EmailVerificationToken unique (tokenHash). RefreshToken unique (tokenHash), (userId). DeviceToken unique (token), (userId).

## 5.3 Channel, ChannelMember, Message

Common rules of section 5.1 apply (int auto-increment ids, timestamptz truncated to milliseconds, enums stored as UPPER_SNAKE_CASE strings, camelCase names). "FK → X" is a foreign key to X.id, "?" means nullable.

### Channel

One row per channel of a team. Every team has exactly one default channel (#general), created together with the team. type and isDefault never change after creation. On delete: CASCADE from Team.

| Field | Type | Constraints / description |
|---|---|---|
| id | int | PK, auto-increment |
| teamId | int | FK Team.id |
| name | varchar(50) | 1-50 characters, stored trimmed, unique (teamId, lower(name)) |
| type | enum | PUBLIC, PRIVATE (set at creation, #general is PUBLIC) |
| isDefault | bool | Default false. True only for #general, at most one row per team (partially unique index) |
| createdAt | timestamptz | - |
| createdById | int | FK User.id; never null |

### ChannelMember

One row per user of a private channel. Public channels have no rows, because everyone in the team can view them through TeamMember. Rows are created with the channel (the creator and the initial members) and by adding a member, and they are deleted by removing a member, when the member leaves the channel, or when the user leaves or is removed from the team. On delete: CASCADE from Channel, User.

| Field | Type | Constraints / description |
|---|---|---|
| id | int | PK, auto-increment |
| channelId | int | FK Channel.id, the channel must have type = private (checked by the service) |

| Field | Type | Constraints / description |
|---|---|---|
| userId | int | FK User.id must be a member of the channel's team (checked by the service). Unique (channelId, userId). The user's displayName and tag are read from User and are not copied here. |
| createdAt | timestamptz | When the user was added |

**Message**

One row per message of a channel, user message or system message. A deleted message stays as a row (soft delete): content is set to null and deletedAt is filled, so replies keep their quote and notifications can still open "Message deleted". Messages are removed for good only together with their channel. On delete: CASCADE from Channel; authorId and parentId are not changed by user or message deletion (users are not deleted, and a reply is always in the same channel as its parent).

| Field | Type | Constraints / description |
|---|---|---|
| id | int | PK, auto-increment |
| channelId | int | FK Channel.id |
| type | enum | USER, SYSTEM |
| authorId | int, nullable | FK User.id, null when type = SYSTEM, required when type = USER |
| content | text, nullable | Plain text, 1-4000 characters after trimming. For SYSTEM messages the final text (e.g. "@rokas joined the team"). Null after deletion |
| clientMessageId | varchar(64), nullable | Idempotency key sent by the client, required when type = USER, null for SYSTEM. Unique (authorId, clientMessageId) |
| parentId | int, nullable | FK Message.id, the message replied to, in the same channel (checked by the service), cannot be changed after creation |
| editCount | smallint | Default 0, at most 5 |
| editedAt | timestamptz, nullable | Time of the last edit (null by default or if message was deleted) |
| deletedAt | timestamptz, nullable | Set when the message is deleted (isDeleted in the API is deletedAt IS NOT NULL) |
| pinnedAt | timestamptz, nullable | Set when the message is pinned (isPinned in the API is pinnedAt IS NOT NULL) |
| pinnedById | int, nullable | FK User.id, filled together with pinnedAt, cleared also with it |
| createdAt | timestamptz | Messages are ordered by (createdAt, id), NEVER BY ID ALONE |

**Indexes.** Channel unique (teamId, lower(name)), partial unique (teamId) where isDefault = true. ChannelMember unique (channelId, userId), (userId). Message (channelId, createdAt, id), (channelId, pinnedAt desc, id desc) where pinnedAt is not null, partial unique (authorId, clientMessageId) where clientMessageId is not null, (parentId) where parentId is not null, GIN trigram on content (pg_trgm).

**Relationships.** Team 1 - * Channel; Channel 1 - * ChannelMember (private channels only); User 1 - * ChannelMember; Channel 1 - * Message; User 1 - * Message (author, pinned by); Message 1 - * Message (replies through `parentId`).

## 5.4 Reaction, Mention, Notification, ReadState

### Reaction

One row per user, message and emoji; rows of deleted messages are kept but not returned. On delete: CASCADE from Message, User.

| Field | Type | Constraints / description |
|---|---|---|
| `id` | int | PK, auto-increment |
| `messageId` | int | FK Message.id |
| `userId` | int | FK User.id |
| `emoji` | varchar(32) | One emoji (one grapheme cluster), max 32 Unicode code points; unique (`messageId`, `userId`, `emoji`) |
| `createdAt` | timestamptz | |

### Mention

Saved by the server when a message is sent or edited. On delete: CASCADE from Message, User.

| Field | Type | Constraints / description |
|---|---|---|
| `id` | int | PK, auto-increment |
| `messageId` | int | FK Message.id |
| `type` | enum | `USER`, `CHANNEL` |
| `mentionedUserId` | int, nullable | FK User.id; required when `type = USER`, null when `type = CHANNEL`; unique (`messageId`, `mentionedUserId`); at most one CHANNEL row per message (partial unique index on `messageId` where `type = CHANNEL`) |
| `ranges` | jsonb | Where the mention is in `content`: `[{ start, length }]` in UTF-16 code units, one item per occurrence; recomputed on edit |
| `createdAt` | timestamptz | |

### Notification

One row per recipient; preview is computed, not stored. On delete: CASCADE from User, Team, Channel, Message, Task; `actorId` SET NULL.

| Field | Type | Constraints / description |
|---|---|---|
| `id` | int | PK, auto-increment |
| `userId` | int | FK User.id (recipient) |
| `teamId` | int | FK Team.id |
| `type` | enum | `MENTION`, `CHANNEL_MENTION`, `REPLY`, `TASK_ASSIGNED`, `TASK_STATUS_CHANGED` |

| Field | Type | Constraints / description |
|---|---|---|
| actorId | int, nullable | FK User.id (who caused it); null if caused by the system |
| channelId | int, nullable | FK Channel.id; filled for MENTION, CHANNEL_MENTION, REPLY |
| messageId | int, nullable | FK Message.id; filled for MENTION, CHANNEL_MENTION, REPLY; unique (userId, messageId) |
| taskId | int, nullable | FK Task.id; filled for TASK_ASSIGNED, TASK_STATUS_CHANGED |
| taskStatus | enum, nullable | Task status after the change; filled for TASK_STATUS_CHANGED |
| isRead | bool | Default false |
| readAt | timestamptz, nullable | |
| createdAt | timestamptz | |

**ReadState**

One row per user and viewable channel. On delete: CASCADE from User, Channel; lastReadMessageId SET NULL (lastReadAt stays).

| Field | Type | Constraints / description |
|---|---|---|
| id | int | PK, auto-increment |
| userId | int | FK User.id |
| channelId | int | FK Channel.id; unique (userId, channelId) |
| lastReadMessageId | int, nullable | FK Message.id; null if the channel had no messages when the row was created |
| lastReadAt | timestamptz | createdAt of lastReadMessageId, or the time the user got access if it is null |
| updatedAt | timestamptz | |

**Indexes.** Reaction unique (messageId, userId, emoji). Mention unique (messageId, mentionedUserId), (mentionedUserId), partial unique (messageId) where type = CHANNEL. Notification (userId, teamId, createdAt desc, id desc), (userId, teamId) where isRead = false, unique (userId, messageId). ReadState unique (userId, channelId). Message (channelId, createdAt, id), (channelId, pinnedAt desc, id desc) where pinnedAt is not null, GIN trigram on content (pg_trgm).

# 6 Permissions matrix

## 6.1 Role x action matrix (owner/admin/member)

The system has three team-level roles: Owner, Admin and Member. Each team has exactly one Owner: the user who created it, until ownership is transferred. Permissions are checked on every REST request and WebSocket action. A forbidden action returns 403 Forbidden with error code FORBIDDEN.

**Team management**

| Action | Owner | Admin | Member |
|---|---|---|---|
| Edit team name and settings | Yes | Yes | No |
| Delete team | Yes | No | No |
| Create invite code | Yes | Yes | No |
| Revoke invite code | Yes | Yes | No |
| Join with invite code | - | - | Yes, any registered user |
| Remove member from team | Yes | No | No |
| Change member roles (promote / demote) | Yes | No | No |
| Transfer ownership | Yes (to an Admin) | No | No |
| Claim ownership when the Owner is inactive for 30 days | - | Longest-serving Admin | Longest-serving Member, if there are no Admins |
| Leave team | After transferring ownership | Yes | Yes |

**Channels**

| Action | Owner | Admin | Member |
|---|---|---|---|
| View public channels | Yes | Yes | Yes |
| View private channel | Only if channel member | Only if channel member | Only if channel member |
| Create public / private channel | Yes | Yes | No |
| Add / remove members of a private channel | Yes | Yes | No |
| Rename / delete channel (not #general) | Yes | Yes | No |
| Leave private channel | Yes | Yes | Yes |

Owner and Admin may rename, delete and manage the members of any private channel of their team without being a member of it; this does not give them access to its messages.

**Messages**

| Action | Owner | Admin | Member |
|---|---|---|---|
| Send message, reply, react, mention | Yes | Yes | Yes |
| Edit own message (at most 5 edits) | Yes | Yes | Yes |
| Delete own message | Yes | Yes | Yes |
| Delete another user's message | Yes | Yes | No |
| Notify whole channel (@channel) | Yes | Yes | No |

| Action | Owner | Admin | Member |
|---|---|---|---|
| Pin / unpin message | Yes | Yes | No |

**Tasks**

| Action | Owner | Admin | Member |
|---|---|---|---|
| Create task | Yes | Yes | No |
| Assign task to Admin | Yes | No | No |
| Assign task to Member | Yes | Yes (tasks they created) | No |
| Self-assign an open task | Yes | Yes | Yes |
| Edit / delete task | Yes | Task they created | No |
| Change status OPEN -> IN_PROGRESS -> PENDING_REVIEW | Yes | Yes | Assignee only |
| Approve PENDING_REVIEW -> DONE or return with comment PENDING_REVIEW -> IN_PROGRESS | Yes | Yes | No |
| Unassign / reassign task | Yes | Yes (tasks they created) | No |
| Abandon an assigned task | No | No | No |

Permissions are identical for web and mobile clients. A role change takes effect immediately: the server checks the current role on every request, and clients update the visible controls when they receive `member.role_changed`.

# 7 API contract

**API contract:** View in Swagger Editor (endpoints, rules, schemas)

## 7.1 WebSocket - Connection, authentication, and event format

The WebSocket carries real-time events from the server to the clients (both web and mobile). Real-time events such as: new messages, reactions, notifications, presence, typing. Three important principles:

- Server is the only sender of events. Clients change state through REST. The only messages a client sends over the WebSocket are the typing signals (see 7.11).
- An event only tells the client that something changed. REST API stays the source of truth and missed events are not replayed (see Reconnecting).
- A client that cannot use the WebSocket still works through REST, only without live updates.

**Connection**

| Rule | Value |
|---|---|
| URL | `wss://{host}/api/v1/ws` (WebSocket upgrade, HTTPS only) |
| Frames | Text frames only. One JSON object (RFC 8259 UTF-8) per frame. Binary frames are not used. |
| Connections per user | Any number (several tabs, web and mobile at the same time). Each is a separate connection with its own `connectionId`. |
| Maximum size of a client message | 4 KiB, a larger frame closes the connection with code 1009. |
| Heartbeat | Server sends a WebSocket ping every 15 s. A connection that has been silent for 30 s is considered lost (OFFLINE), and the server removes it. Client applies the same 30 s rule and reconnects. |

### Authentication

WebSocket uses the same JWT as REST (see 7.2). Browsers cannot set an `Authorization` header on a WebSocket request, so the token is sent in the query string:

```
wss://app.example.com/api/v1/ws?access_token=<jwt>
```

Native clients (the mobile app) may send `Authorization: Bearer <token>` instead. The server accepts both. The server must not write the query string of this URL to its logs.

If a token is missing, malformed, or expired at connect time, there is no upgrade and the server answers `HTTP 401` with the error body of section 7.3. If the token expires while the connection is open, the server closes the connection with code 4401; the client obtains a new token and reconnects.

### Connection sequence

1. The client opens the WebSocket with its token.
2. The server validates the token. If it is invalid, the handshake fails (see above).
3. The server registers the connection (userId to connectionIds) and adds it to the groups of the user (see the group table in WebSocket - Reaction, notification and task events).
4. The server sends `connection.ready`. Only now are the groups guaranteed to be joined.
5. The client loads the current state over REST (see Reconnecting).

The client must wait for `connection.ready` before loading REST data. Otherwise an event sent between the REST request and the group join will be lost.

```
{ "event": "connection.ready", "occurredAt": "2026-10-08T13:47:05.120Z",
  "data": { "connectionId": "c_8f3a2b", "userId": 7 } }
```

Payload of `connection.ready` (`data`):

| Field | Type | Description |
|---|---|---|
| `connectionId` | string | Identifier of this connection, generated by the server (e.g. `c_8f3a2b`). Unique among the open connections and different for every new connection, also for the same user. Opaque: it is not an entity identifier, is never used in URLs, and the client does not interpret it. Useful for logging and for support. |
| `userId` | integer | Id of the user the token belongs to (`sub` of the JWT). The client can use it to recognize its own events (e.g. `presence.changed` or `typing.started` of the same user from another connection) |

### Event format (server to client)

Every event is one JSON object with the same three fields:

| Field | Type | Description |
|---|---|---|
| `event` | string | Event name, `resource.action`, lowercase (f.e. `message.created`) |
| `occurredAt` | string (ISO 8601 UTC) | When the server produced the event, with milliseconds and a `Z` suffix |
| `data` | object | Payload of the event. Always an object, never null. Defined per event in sections 7.11 and 7.12. |

Rules:

- Fields are camelCase. IDs are integers. Enums are UPPER_SNAKE_CASE strings. All of this is the same as in REST (see 7.1).
- Events are sent after the change has been saved. A REST response and the matching event may reach a client in either order, so clients must handle both orders and ignore duplicates (match by message ID or `clientMessageId`).
- Delivery is at most once. Events of one group arrive in the order in which the server published them. There is no ordering between different groups.
- Clients must ignore events and fields they do not know. New events and fields can be added without changing the version.

### Messages from the client

Clients send only typing signals. Every message has the exact same template (shape):

```
{ "action": "typing.start", "data": { "channelId": 12 } }
```

| Field | Type | Description |
|---|---|---|
| `action` | string | Action name f.e. `resource.action` (lowercase) |
| `data` | object | Parameters of the action |

The server does not confirm a successful action. It answers only with `connection.error` (if an error occurs) and keeps the connection open:

```
{
  "event": "connection.error",
  "occurredAt": "2026-10-08T13:47:05.120Z",
  "data": {
    "code": "CHANNEL_NOT_FOUND",
    "message": "Channel not found.",
    "action": "typing.start"
  }
}
```

Some examples of possible error codes:

| Code | When |
|---|---|
| `VALIDATION_ERROR` | The frame is not valid JSON, the action is unknown, or `channelId` is not a positive integer |
| `CHANNEL_NOT_FOUND` | The channel does not exist, or the user cannot view it (hidden on purpose, as in REST) |
| `FORBIDDEN` | The role does not allow the action (no current WebSocket action is role-restricted; roles are changed through REST only) |

**Closing and reconnecting**

| Close code | Meaning | What client does |
|---|---|---|
| 1000 | Normal closure (client left) | Nothing |
| 1001 | The server is going away (restart, deployment) | Reconnect with backoff |
| 1008 | Policy violation | Reconnect after a delay and fix the cause |
| 1009 | Message too big | Do not resend. Reconnect |
| 1011 | Internal server error | Reconnect with backoff |
| 4401 | Token expired or revoked | Get a new token, then reconnect |

Reconnecting:

- The client reconnects after a lost connection with exponential backoff: 1 s, 2 s, 5 s, 10 s, then 30 s at most, each with up to 20% random jitter. The backoff is reset after `connection.ready`.
- Missed events are not replayed. After every connect, including the first one, the client waits for `connection.ready` and then reloads: the unread counts, the notifications of the team, the current presence of the team, and the open channel (see WebSocket - Messaging, presence and typing events and REST - Replies, reactions, pins, notifications, search).
- A mobile app closes the connection when it moves to the background and reconnects when it returns to the foreground. A web page that is only hidden keeps its connection.

## 7.2 WebSocket - Messaging, presence and typing events

Uses the standard connection and event format (see 7.10). Events trigger after successful persistence. Group mappings (`channel-{channelId}`, `user-{userId}`, `team-{teamId}`) are defined in 7.12.

| Event (group) | Payload | Notes |
|---|---|---|
| `message.created` (channel) | `channelId`, `message`, `clientMessageId` | Sent to all channel connections (author matches via `clientMessageId`). Drives unread badges. For system messages the author is null. |
| `message.updated` (channel) | `channelId`, `message` | Replaces full message content (updated object). |
| `message.deleted` (channel) | `channelId`, `message` | (`isDeleted = true`, hidden content, empty reactions) |
| `channel.created` (team/user) | `channel` | Broadcast to `team-{teamId}` if public, or `user-{userId}` for initial members if private. |
| `channel.updated` (channel) | `channel` | Updates channel metadata. |
| `channel.deleted` (channel) | `channelId`, `teamId` | Sent before group dissolution. |
| `channel.joined` / `channel.left` (user) | `channel` / `channelId` | Sent to `user-{userId}` when added to / removed from a private channel, or when the user leaves it. |

**Presence**

| Event (group) | Payload | Behavior |
|---|---|---|
| `presence.changed` (team) | `teamId`, `userId`, `status` (ONLINE/OFFLINE), `availability` (string/null) | ONLINE on first connection, OFFLINE 5 s after last connection closes. Availability changes trigger events even if status is unchanged. Sent to the user's team groups. Background mobile apps count as OFFLINE. |

Initial state load (REST): `GET /teams/{teamId}/presence` → `200 { items: [{ id, tag, displayName, status, availability }] }` (lists only ONLINE members; others are OFFLINE).

**Typing**

- Client → Server: `typing.start` (max once per 3 s while typing) and `typing.stop` (sent on clear/close, not on send). Payload: `{ channelId }`.
- Server → Client: `typing.started` (`channelId`, `user { id, tag, displayName }`) and `typing.stopped` (`channelId`, `userId`). Broadcast to `channel-{channelId}` except sender.

Rules: the server throttles `typing.start` to once per 2 s per user/channel. Clients display indicators with a 5 s reset timer. Clears on `typing.stopped`, matching `message.created`, or timer expiration. Events are ephemeral (never stored/replayed).

**Event payload examples**

```
[
  { "event": "message.created", "occurredAt": "2026-10-08T13:47:05.120Z",
    "data": {
      "channelId": 12,
      "clientMessageId": "6f1c2a9e-3b7d-4c55-9a0e-2f8d1e4b7a10",
```

```
      "message": {
        "id": 901, "channelId": 12, "type": "USER",
        "author": { "id": 7, "tag": "pijus", "displayName": "Poqkorn" },
        "content": "Please finish your sections",
        "createdAt": "2026-10-08T13:47:05.120Z",
        "editedAt": null, "editCount": 0, "isDeleted": false, "deletedAt": null,
        "parentId": null, "parent": null, "reactions": [], "isPinned": false,
        "mentions": [] } } },
  { "event": "presence.changed", "occurredAt": "2026-10-08T13:47:05.120Z",
    "data": { "teamId": 42, "userId": 9, "status": "ONLINE", "availability": null } },
  { "event": "typing.started", "occurredAt": "2026-10-08T13:47:05.120Z",
    "data": { "channelId": 12,
              "user": { "id": 9, "tag": "rokas", "displayName": "Rokas" } } }
]
```

### 7.3 WebSocket - Reaction, notification and task events

Uses the connection and envelope of WebSocket - Connection, authentication, and event format; only the `data` payload is listed. The server keeps a `userId` → `connectionIds` registry; on connect, and whenever access changes (in the same call that changes `TeamMember` / `ChannelMember`, before any system message), it adds or removes all of the user's connections, so a removed user stops receiving that channel at once. Missed events are not replayed: after reconnecting the client reloads `GET unread` and notifications. Events meant only for viewers of the open channel (e.g. typing) carry `channelId` so clients can ignore others.

| Group | Members |
|---|---|
| `channel-{channelId}` | Every connection of every user who can view the channel (receives `message.created`, which drives the badges) |
| `user-{userId}` | All connections of one user |
| `team-{teamId}` | Every connection of every team member |

| Event (group) | Payload |
|---|---|
| `reaction.added` / `reaction.removed` (channel) | `channelId`, `messageId`, `emoji`, `user { id, tag, displayName }`, `count` (new count for this emoji; the client sets `reactedByMe` itself) |
| `message.pinned` (channel) | `channelId`, `message` (pinned message summary), `pinnedBy`, `pinnedAt`, `pinnedCount` |
| `message.unpinned` (channel) | `channelId`, `messageId`, `unpinnedBy` (null if the message was deleted), `latestPin { message, pinnedBy, pinnedAt }` or null, `pinnedCount` |
| `notification.created` (user) | the notification item (as in GET notifications) plus `teamId` and `unreadCount` |
| `notification.read` (user) | `teamId`, `notificationIds` (empty when `allRead`), `allRead`, `unreadCount` |
| `readstate.updated` (user, only when the position moved) | `channelId`, `lastReadMessageId`, `lastReadAt`, `unreadCount` |

| Event (group) | Payload |
|---|---|
| `task.created` / `task.updated` (team) | `task` (task object of REST - Tasks), `changeType` CREATED / EDITED / ASSIGNED / STATUS_CHANGED (EDITED: title, description or dueDate changed through `PATCH /tasks/{taskId}`; ASSIGNED also covers reassign, unassign, take), `changedBy`, `previousStatus` (STATUS_CHANGED only), `previousAssignee { id, tag, displayName }` or null (ASSIGNED only; the new assignee may also be null). The system message about the change is defined in System messages |
| `task.deleted` (team) | `teamId`, `taskId` |

# 8 Open questions and review

No questions are open at the time of submission.